



Modellierung und UML

7. Sequenzdiagramme

Prof. Dr. Michaela Huhn und Prof. Dr. Sebastian Bindick

Gliederung

- **Grundlagen**

- Was ist ein Sequenzdiagramm?
- Anwendungsbereiche von Sequenzdiagrammen
- Abgrenzung zu anderen Diagrammen
- Interaktionsmodellierung

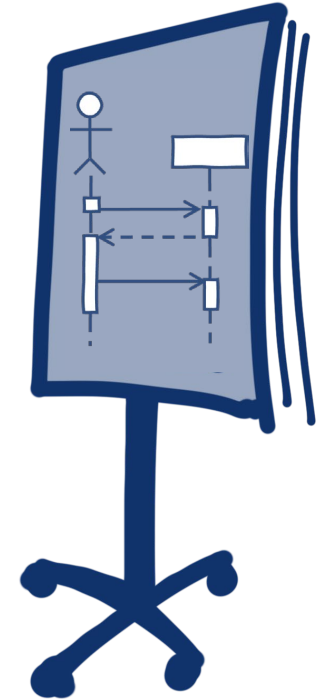
- **UML-Notationselemente im Detail**

- Lebenslinie
- Nachricht
- Interaktionsrahmen
- Kombinierte Fragmente

SEQUENZDIAGRAMME IM ÜBERBLICK

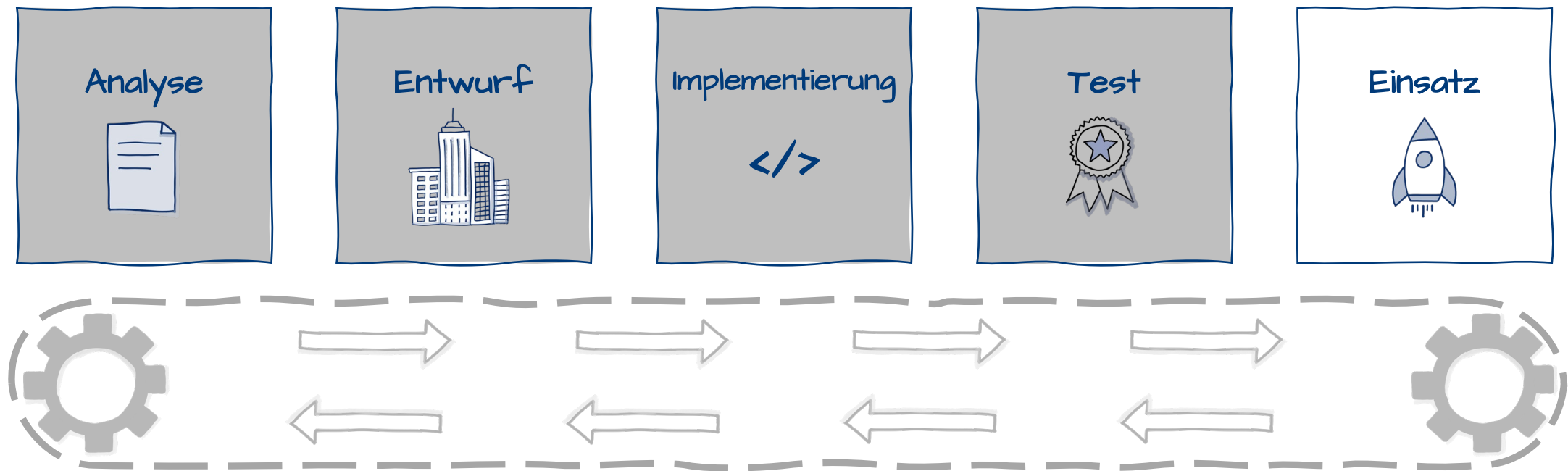
Was ist ein Sequenzdiagramm?

„Sequenzdiagramme **modellieren Interaktionen zwischen Objekten**. Wie Aktivitätsdiagramme stellen auch sie Abläufe dar, konzentrieren sich dabei jedoch auf den **Nachrichtenaustausch** und **nicht** auf die Darstellung aller **möglichen Ablaufpfade**.“ - Kecher*



*Quelle: UML 2.5 - Das umfassende Handbuch,
Kecher, Hoffmann-Elbern, Will, Rheinwerk, 2021

Sequenzdiagramme kommen in den Phasen Analyse, Entwurf, Implementierung und Test zum Einsatz.





Analyse

- Fokus auf die Detaillierung von Anwendungsfällen
- Beschreibung von Nachrichtenflüssen und Interaktionen von Personen und Systemen



Entwurf

- Modellierung der Interaktion einzelner Systembestandteile
- Bsp.: Modellierung der Benutzerinteraktion mit der Benutzungsoberfläche der Software



Implementierung

- Umsetzung der zuvor modellierten Sequenzdiagramme in Quellcode



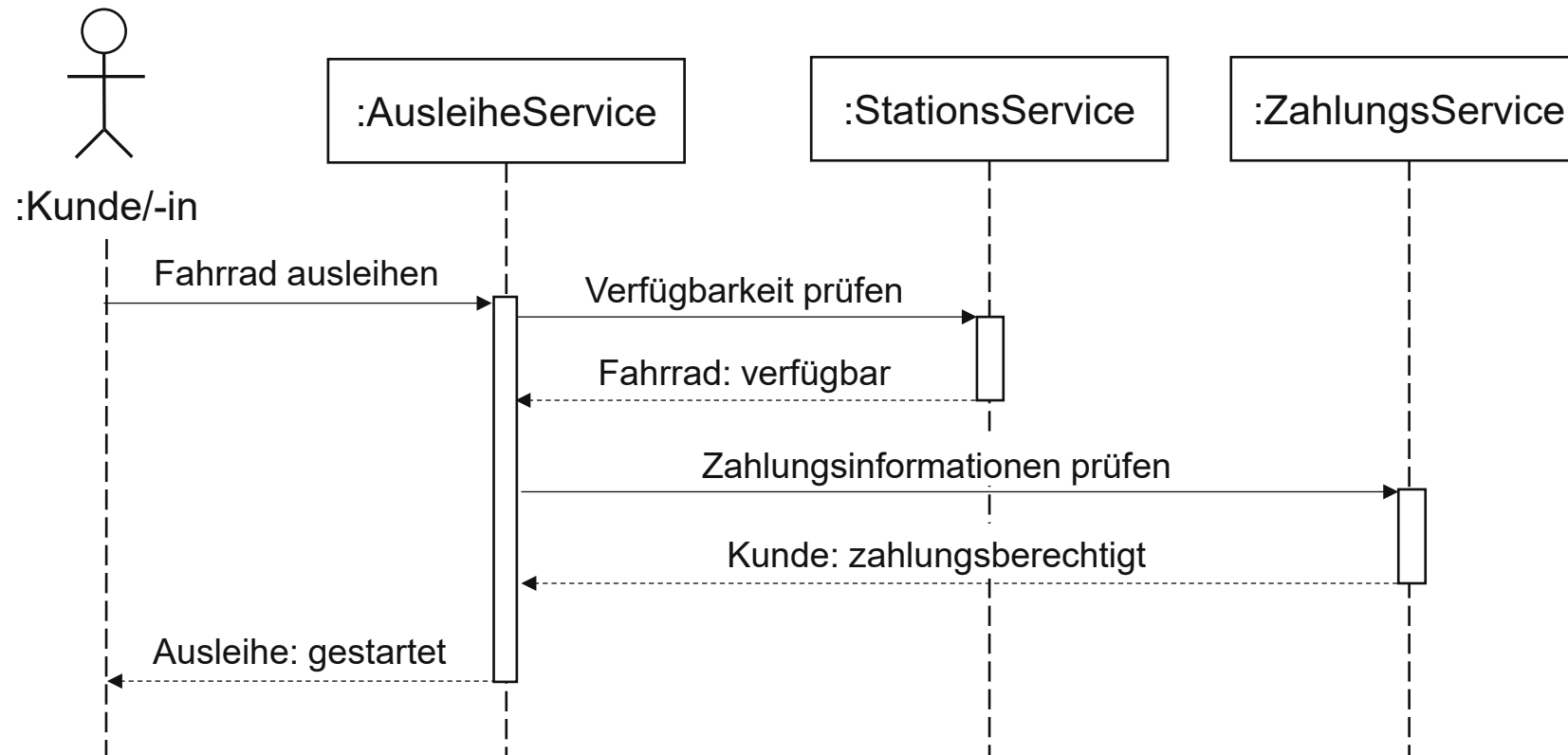
Test

- Grundlage von Testdefinitionen, Modellierung und Dokumentation von Testfällen für Benutzungsoberflächen, Teilsysteme, ...

UML-Sequenzdiagramme im Überblick



Ein **UML-Sequenzdiagramm** zeigt, wie **Objekte** oder **Komponenten** in einem **Ablauf** miteinander **interagieren**, indem es die **zeitliche Reihenfolge** von **Nachrichten** und **Aufrufen** darstellt.



Abgrenzung zu anderen Diagrammen

Sequenzdiagramm	Statechart	Aktivitätsdiagramm
Verhaltensbeschreibung	Verhaltensbeschreibung	Verhaltensbeschreibung
Interaktion mehrerer Objekte	Verhalten eines Objekts	Komplexe Abläufe, nicht zwangsläufig Objektbezogen
exemplarisch	vollständig	vollständig
kein interner Zustand	zustandsbasiert	kein interner Zustand

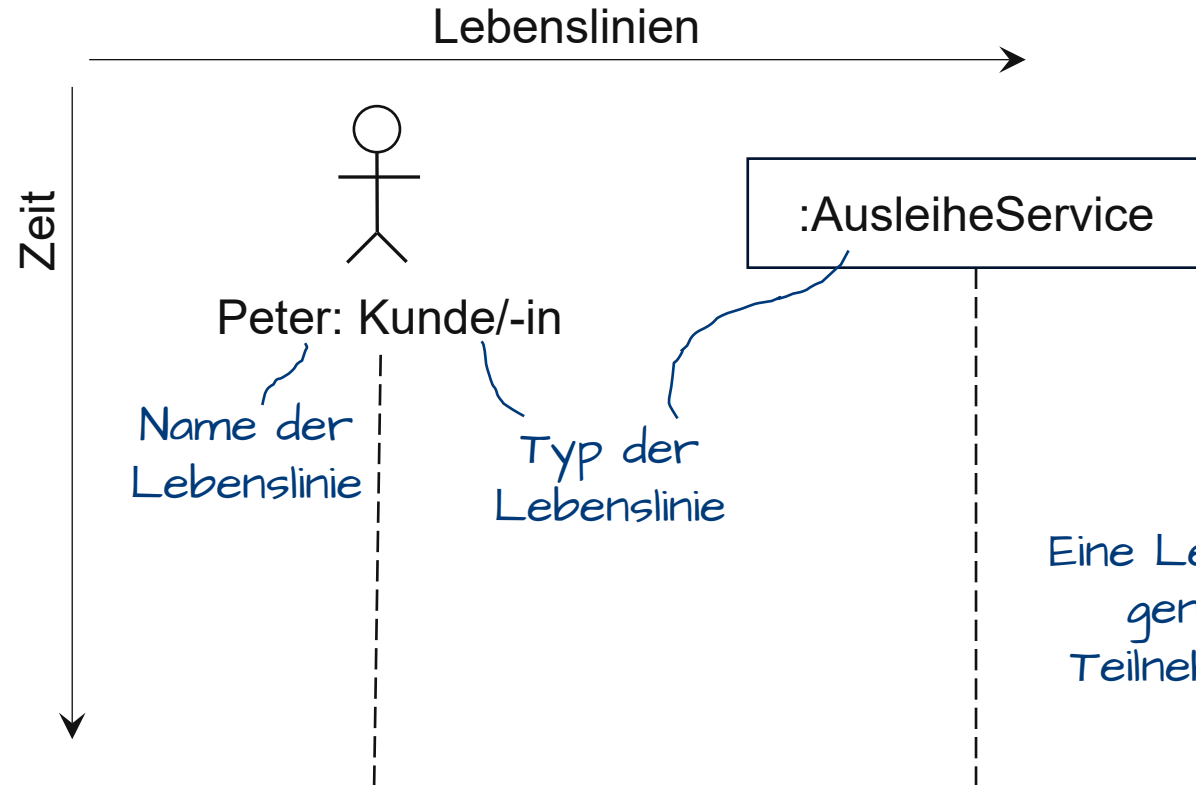
Interaktionsmodellierung

- Sequenzdiagramme sind **nur eine Notation unter vielen zur Darstellung** von Interaktion
- UML-Standard **beschreibt** das **generelle Konzept** (Interaktion, Lebenslinie, Nachricht, Flusskontrolle, ...)
- Konkrete Diagrammarten nur **eine Möglichkeit zum Ausdruck der Interaktion**
 - Nicht alle Konzepte
 - Unterschiedliche Detailtiefe
- **Diagrammarten**
 - Sequenzdiagramme
 - Kommunikationsdiagramme
 - Timing-Diagramme
 - Interaktionsübersichtsdiagramme
- **Sequenzdiagramme aber die wichtigste Spielart**

UML-NOTATIONSELEMENT: LEBENSLINIE

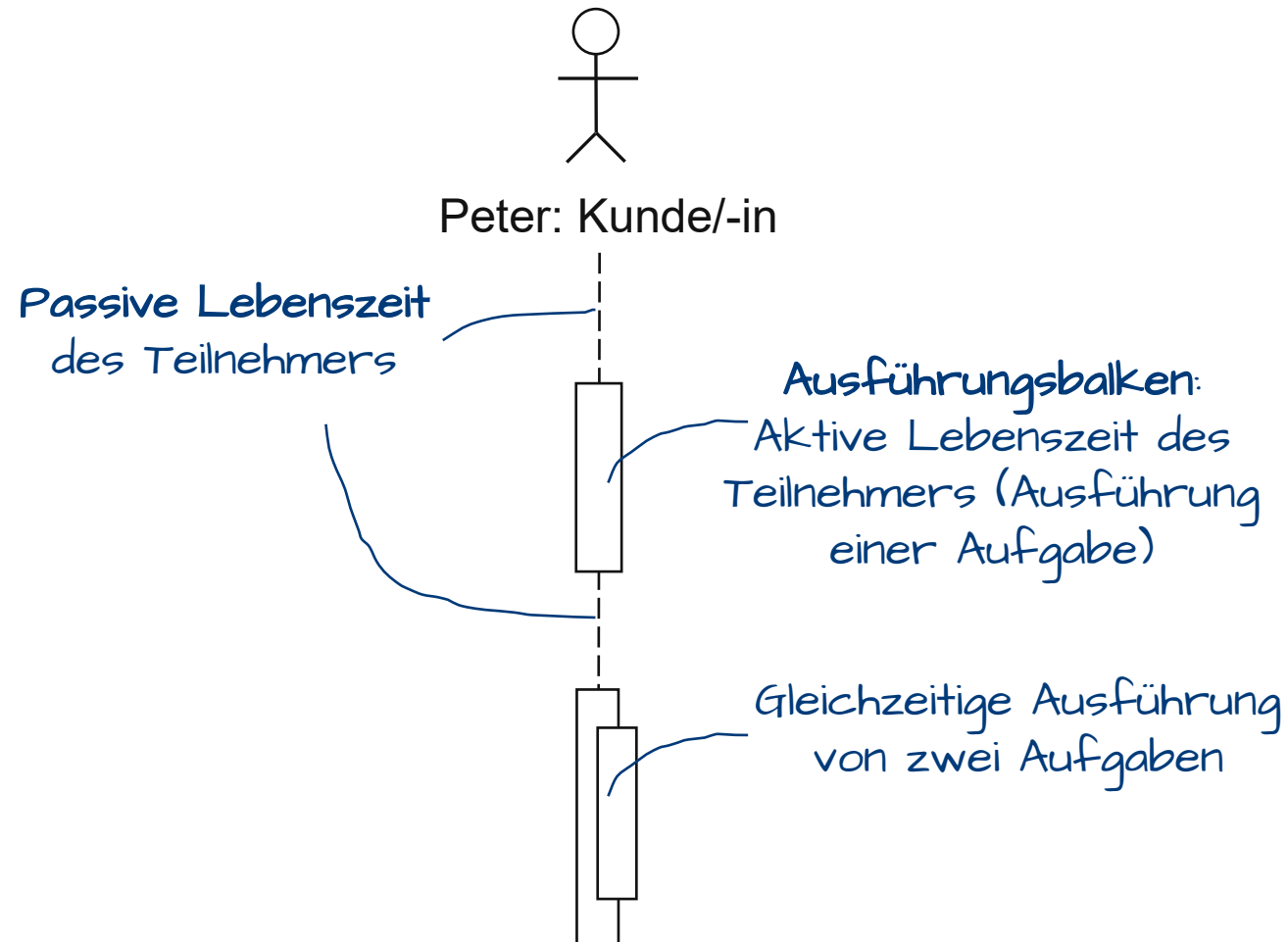
Lebenslinien

Teilnehmer von
Interaktionen werden
durch Lebenslinien
repräsentiert

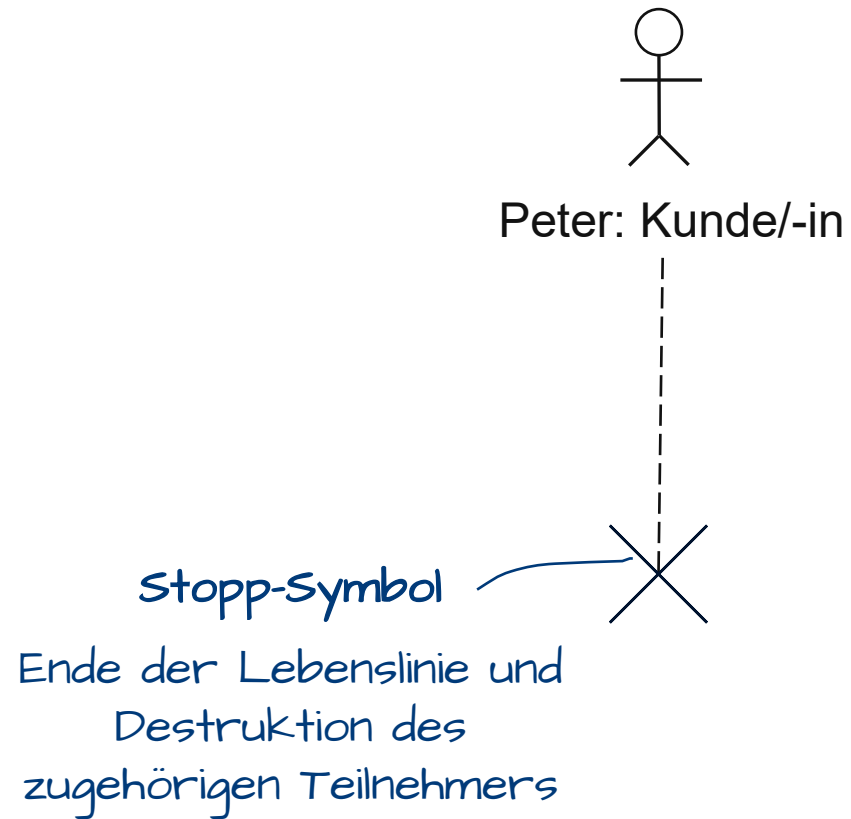


Eine Lebenslinie repräsentiert
genau einen individuellen
Teilnehmer einer Interaktion

Passive und aktive Lebenszeit



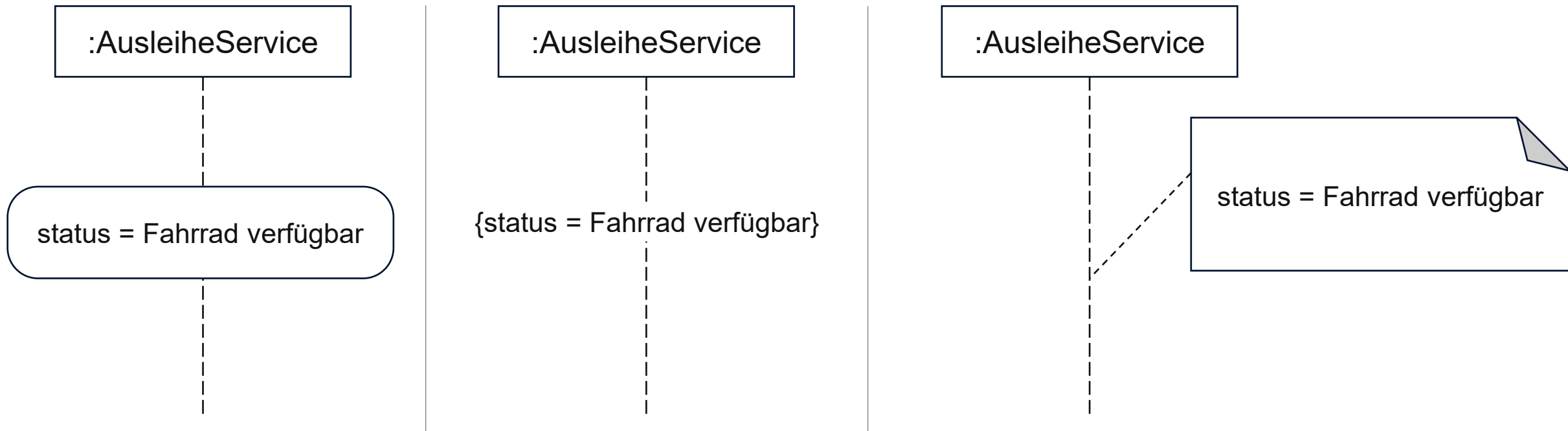
Ende einer Lebenslinie



Zustandsinvarianten von Lebenslinien



Zustandsinvarianten erlauben es, den **Zustand eines Teilnehmers** zu definieren, der für die Teilnahme an der Interaktion erforderlich ist. UML bietet hierzu drei **Notationsmöglichkeiten**.



Dekomposition von Teilnehmern



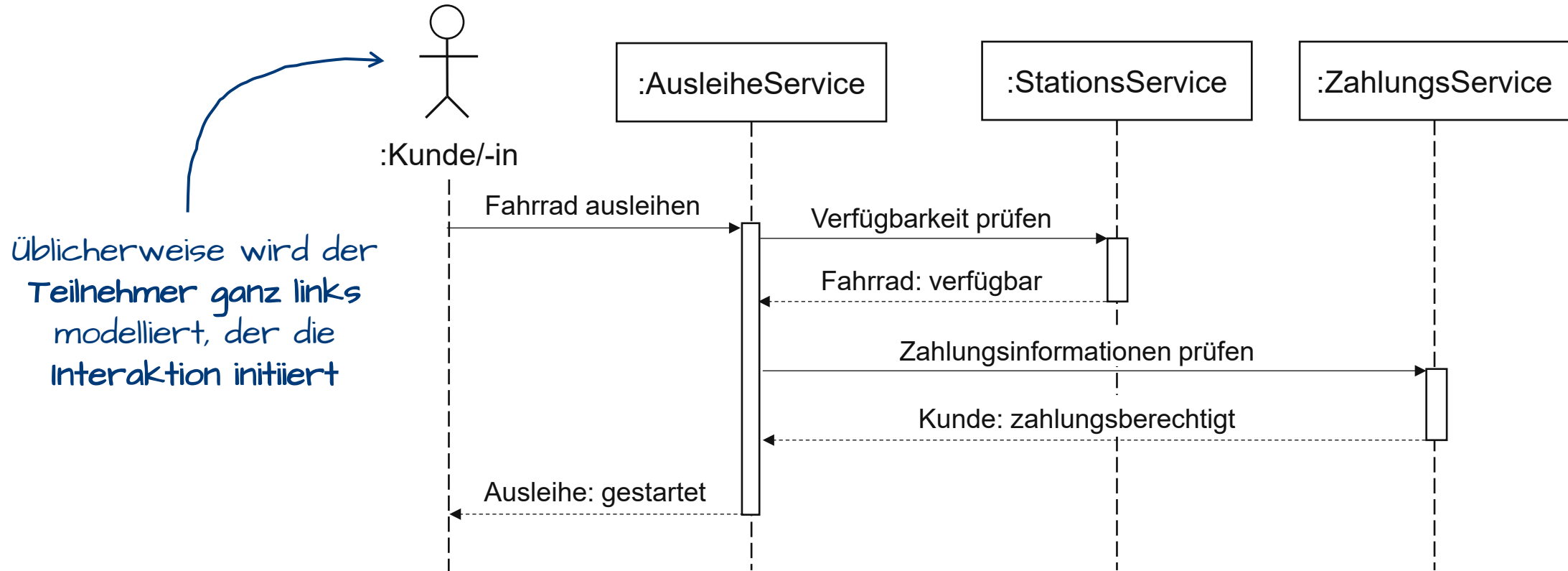
Ein **Teilnehmer** kann durch **Dekomposition** in mehrere interne **Bestandteile** aufgeteilt werden. Dies wird genutzt, wenn die **Interaktionen innerhalb des Teilnehmers wichtig** sind und eine Darstellung als einzelne Einheit im Sequenzdiagramm zu ungenau wäre.



Shop wird in seine
internen Bestandteile
aufgeteilt

Anordnung von Teilnehmern einer Interaktion

Teilnehmer einer Interaktion werden **horizontal nebeneinander** angeordnet



Aufgabe 1: Welche Aussagen treffen zu?

A

Ein Teilnehmer kann mehrere Ausführungsbalken besitzen.

B

Die Zustandsinvariante beschreibt einen Zustand des Fahrrads.

C

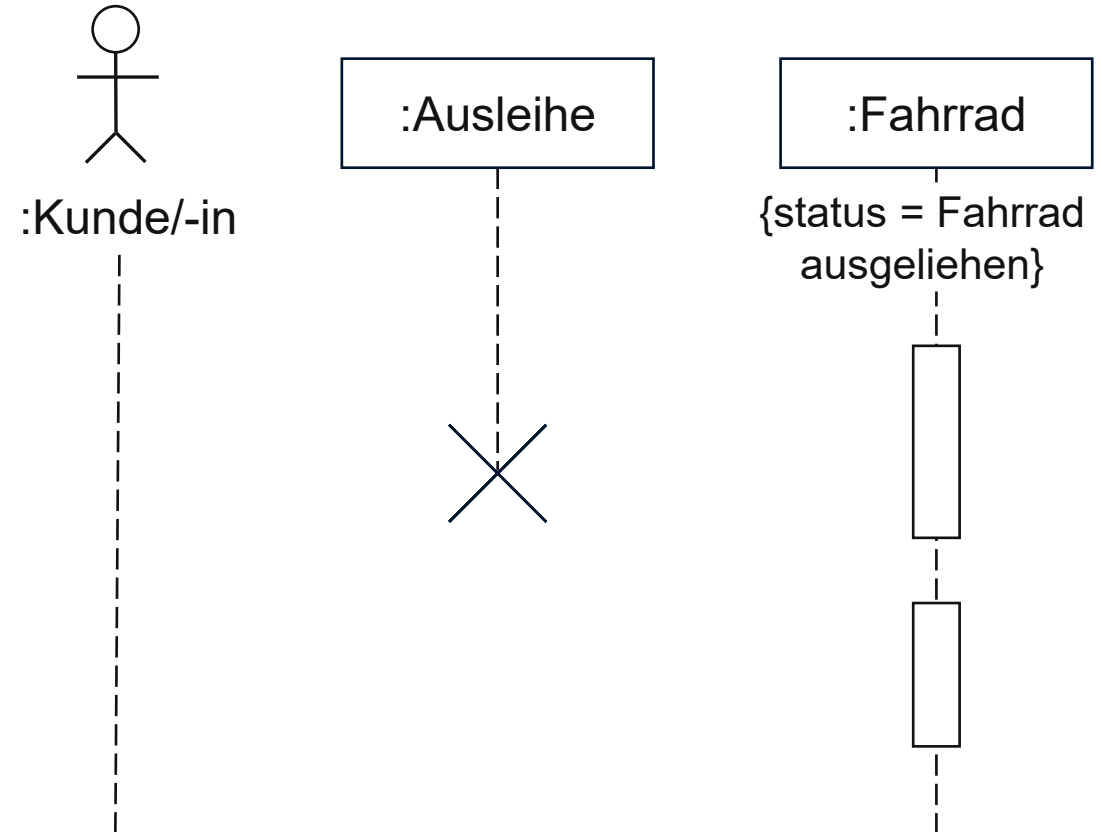
Das Ende der Lebenslinie Ausleihe bedeutet, dass die Instanz nicht mehr existiert.

D

Nach dem Ende einer Lebenslinie kann der Teilnehmer nicht mehr aktiv sein.

E

Der Teilnehmer Fahrrad führt zwei parallele Aufgaben aus.



Aufgabe 2: Welche Aussage beschreibt eine Lebenslinie am besten?

A

Eine Lebenslinie beschreibt eine Klasse im System.

B

Eine Lebenslinie beschreibt eine konkrete Instanz eines Teilnehmers über die Zeit.

C

Eine Lebenslinie beschreibt den gesamten Ablauf eines Systems.

D

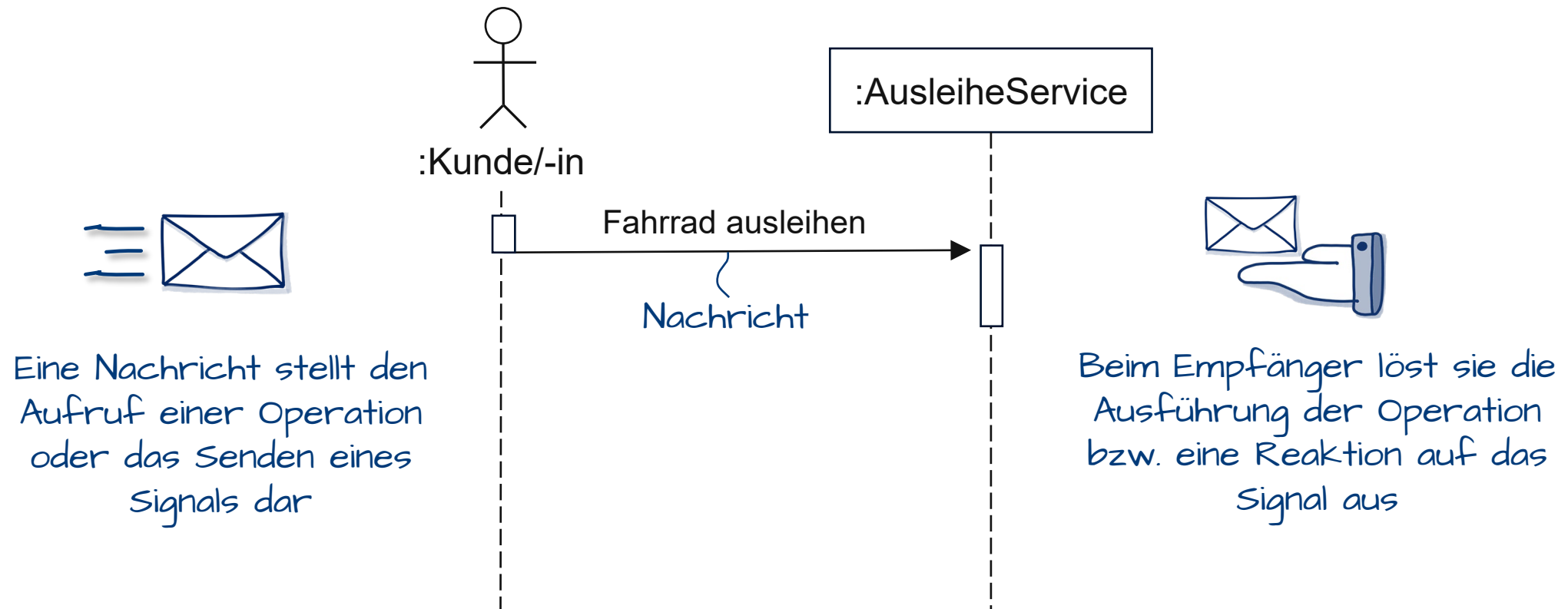
Eine Lebenslinie kann nur genau eine aktive Lebenszeit besitzen.

UML-NOTATIONSELEMENT: NACHRICHT

Nachrichten



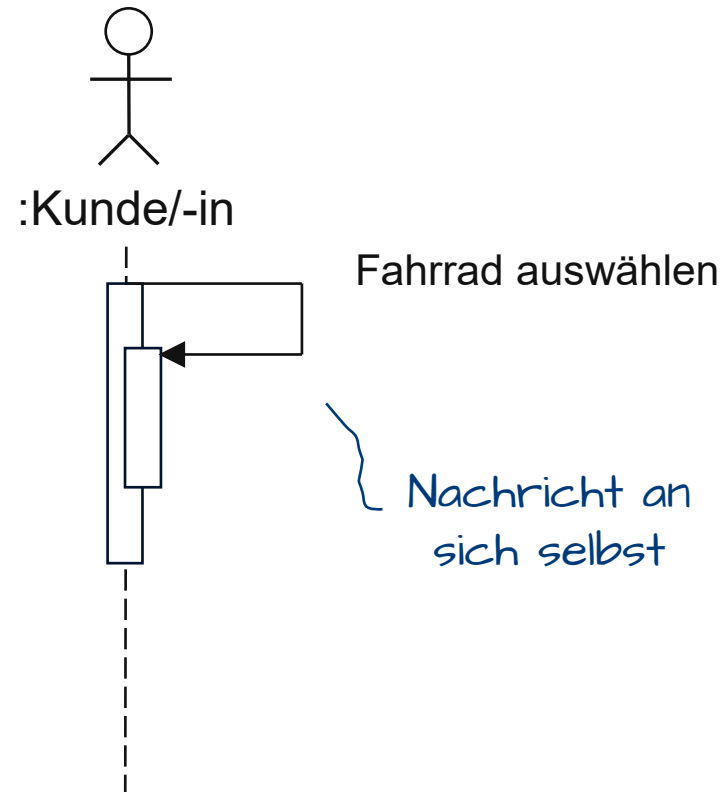
Nachrichten modellieren eine Kommunikation und deren Richtung zwischen zwei Teilnehmern



Nachrichten an sich selbst



Nachrichten können auch **an denselben Teilnehmer gesendet** werden. UML erlaubt somit **Selbstaufrufe** (Nachrichten an sich selbst).



Eine Nachricht an sich selbst (Selbstaufruf) modelliert eine interne Verarbeitung oder Teiloperation eines Teilnehmers.

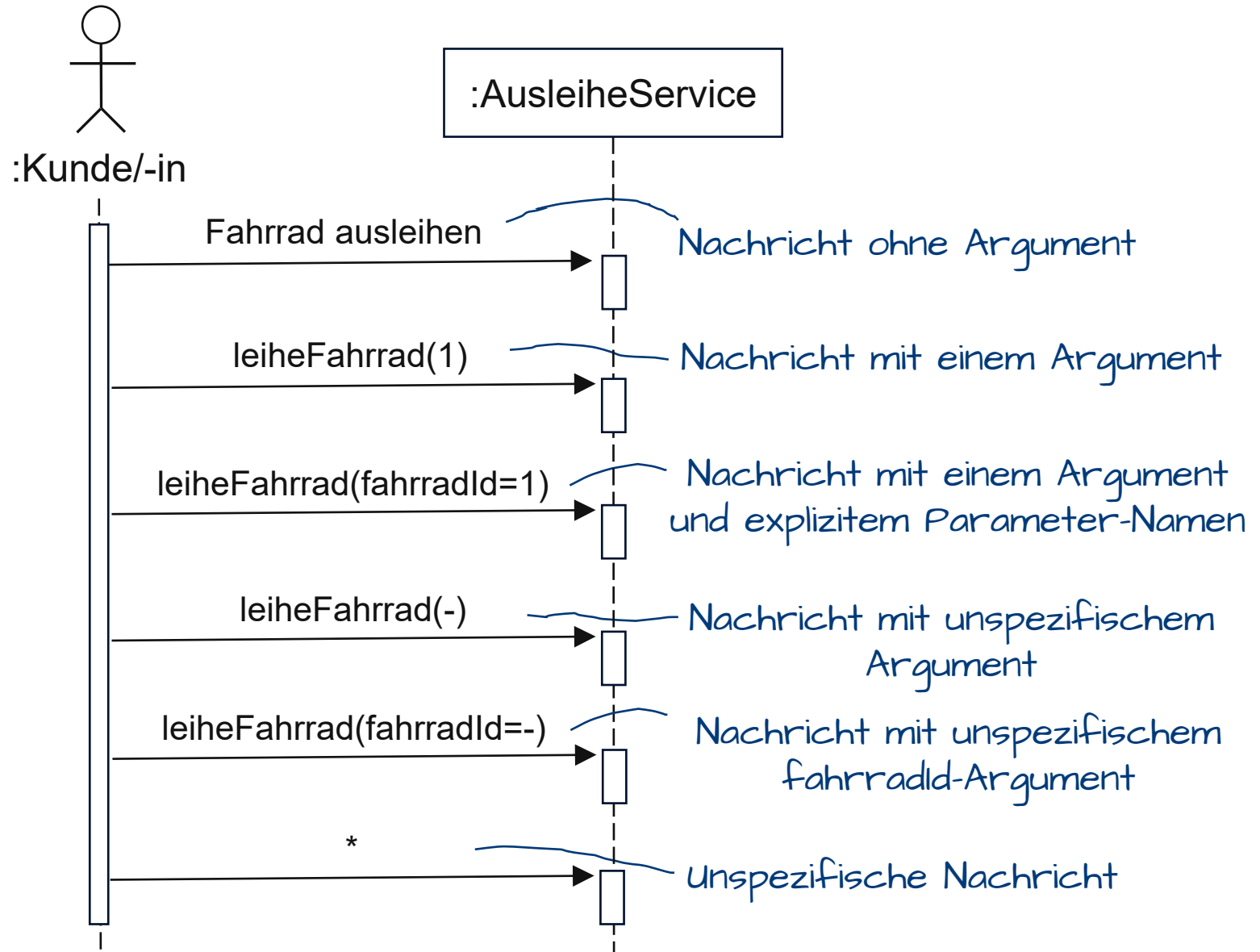
Syntax einer Sende-Nachricht

Syntax:

name [(argument, ...)]

[parameter-name=] argument-wert

oder Bindestrich (-)



Konventionen zur Modellierung von Sende-Nachrichten

Operationsorientierte Notation (empfohlen)

- Nachrichten werden in Sequenzdiagrammen meist als Aufruf einer Operation modelliert
- Empfohlene Notation: Nachrichten als Verb + Objekt (optional mit Argumenten)

BEISPIELE

- `leiheFahrrad(fahrradId)`
- `prüfeVerfügbarkeit(fahrradId)`
- `erstelleAusleihe(kundenId, fahrradId)`

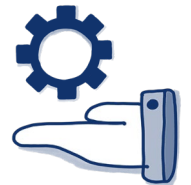
Warum diese Form?

- entspricht einem Operationsaufruf
- ist präzise und eindeutig
- lässt sich gut mit Operationen im Klassendiagramm verbinden



Typischer Einsatz:

- detailliertere Systemmodelle
- Entwurfsphase (Design)



Konventionen zur Modellierung von Sende-Nachrichten

Fachlich-beschreibende Notation

- Nachrichten beschreiben einen fachlichen Schritt

BEISPIELE

- Fahrrad ausleihen
- Verfügbarkeit prüfen
- Ausleihe bestätigen

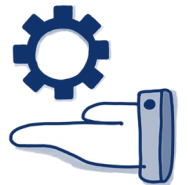
Warum diese Form?

- Fokus liegt auf dem fachlichen Ablauf
- verständlich für fachliche Stakeholder
- keine Festlegung auf konkrete Systemoperationen



Typischer Einsatz:

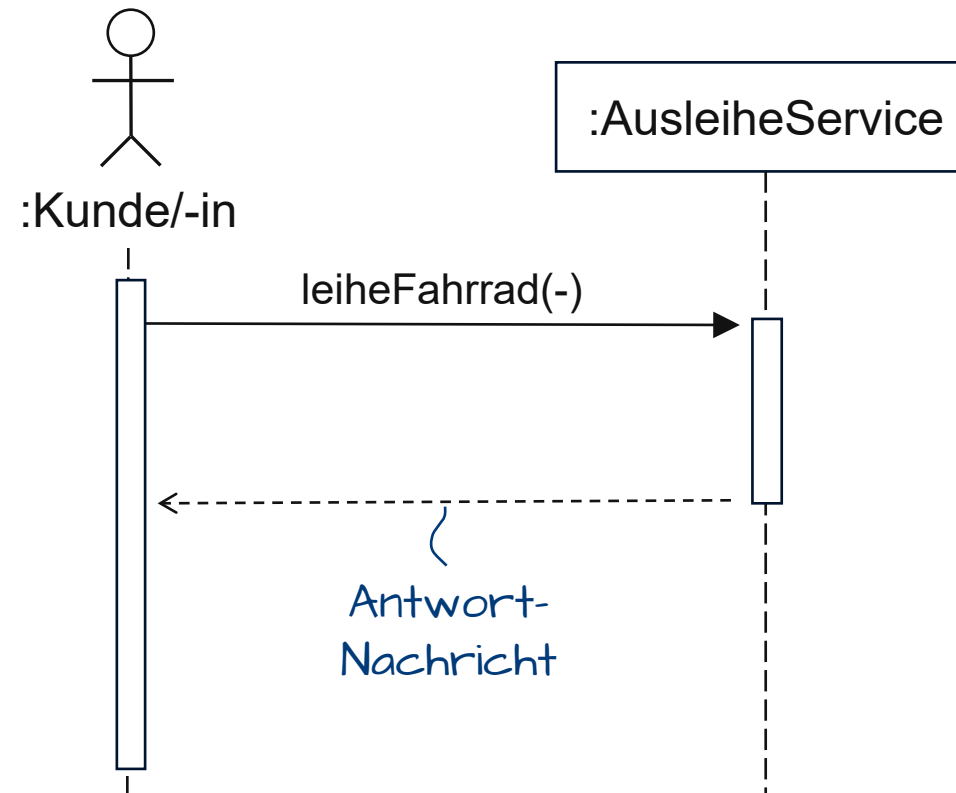
- Use-Case-nahe Darstellung
- frühe Analysephase



Antwort-Nachrichten



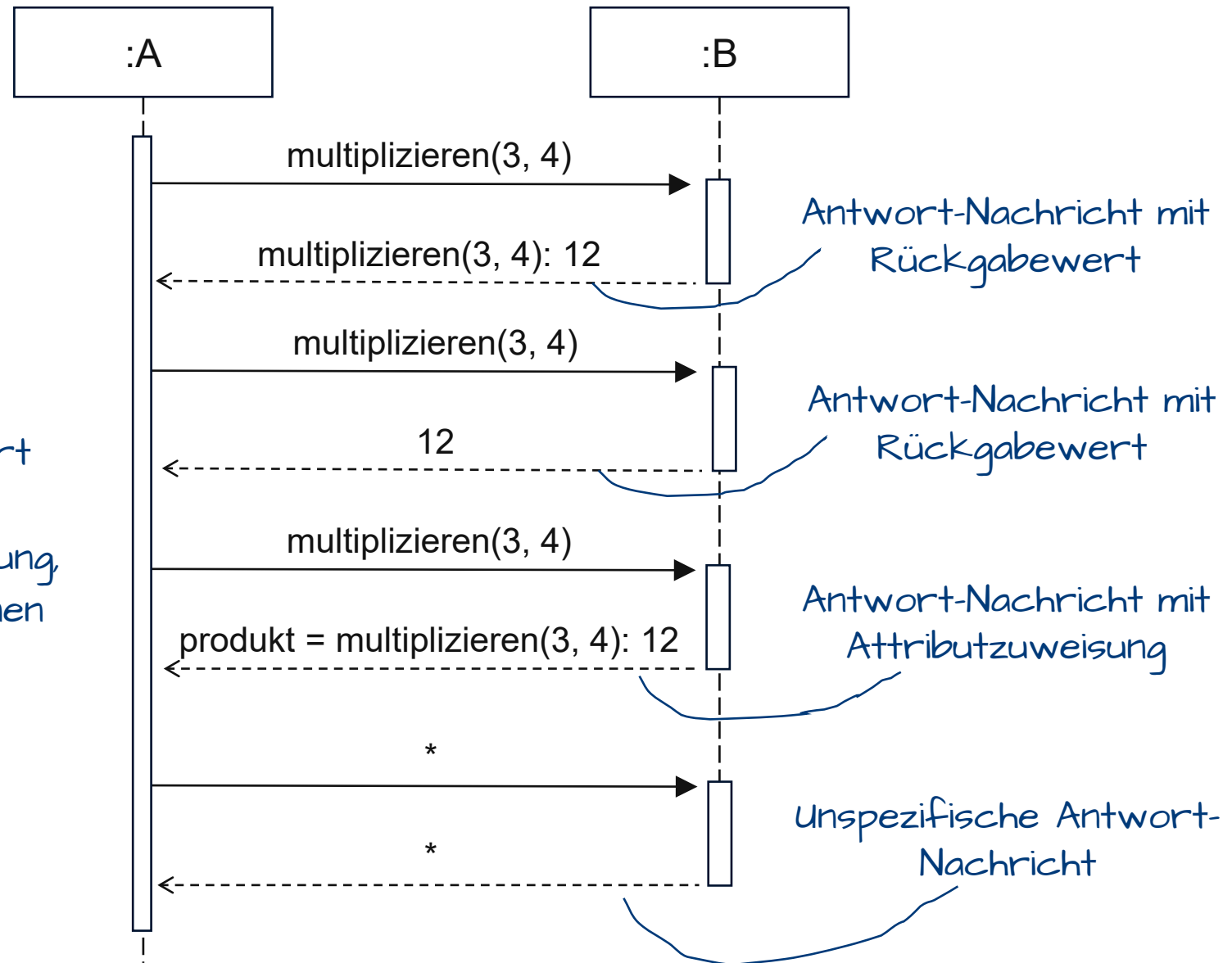
Eine Antwort-Nachricht modelliert die **Rückgabe auf einen Operationsaufruf**. Sie wird mit einem **gestrichelten Pfeil** dargestellt und kann die **Bezeichnung der ursprünglichen Nachricht sowie Rückgabewerte** enthalten.



Rückgabewerte



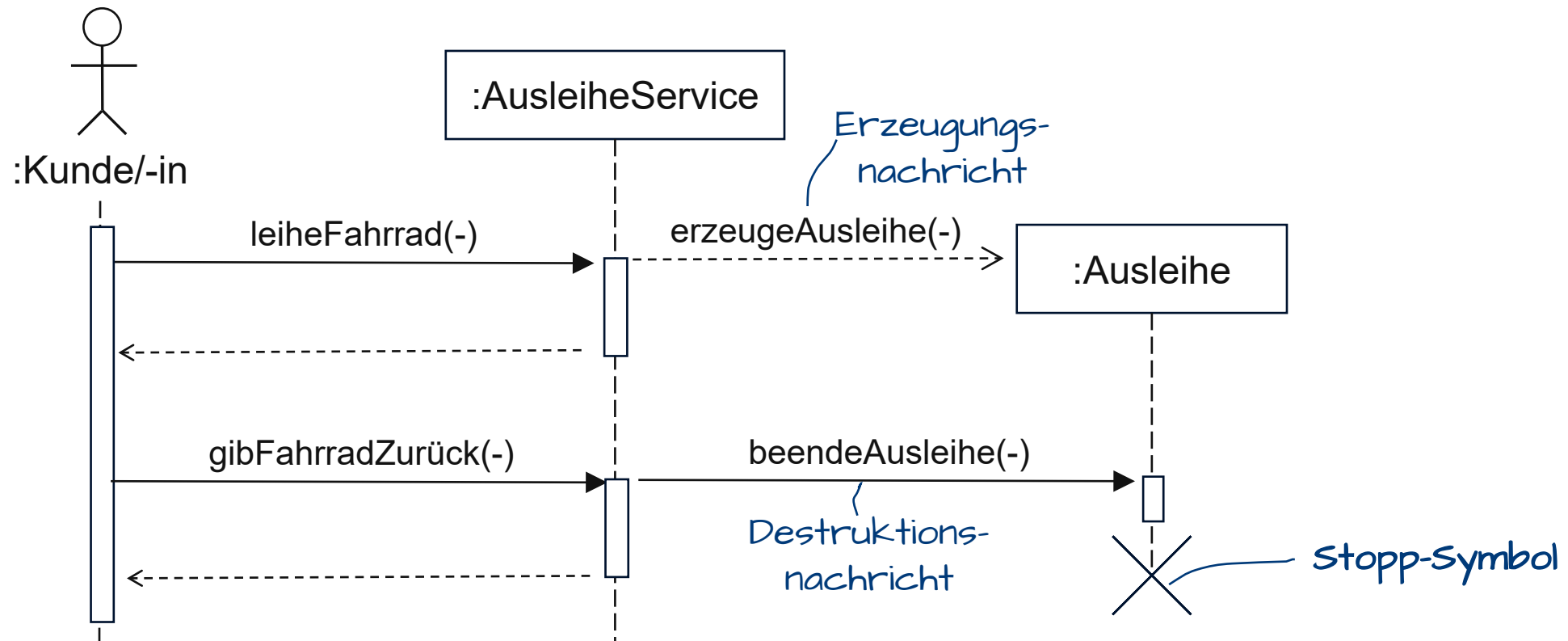
Der Rückgabewert
folgt der
Operationsbezeichnung,
getrennt durch einen
Doppelpunkt



Erzeugungs- und Destruktionsnachricht

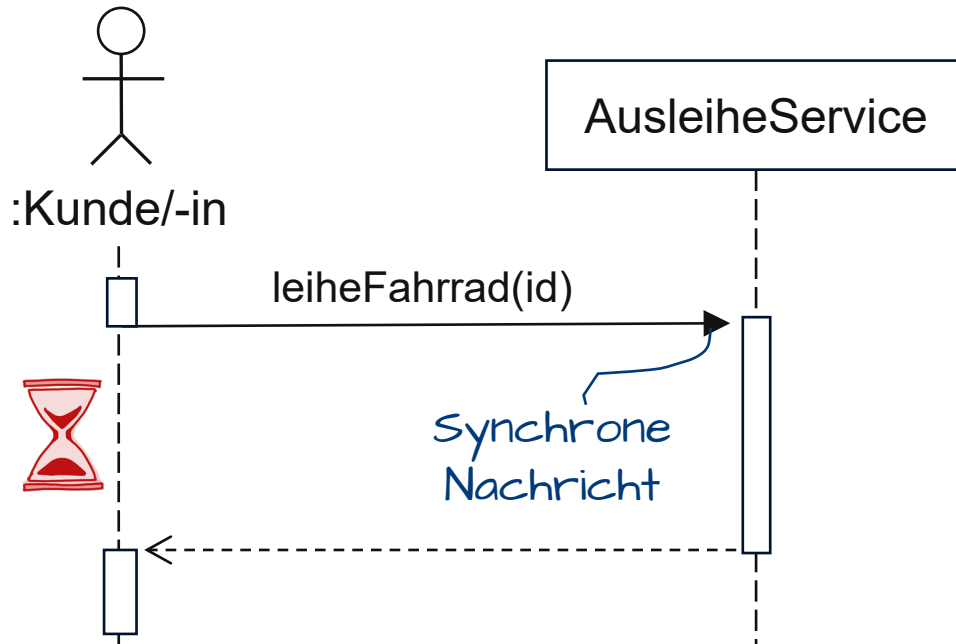


Teilnehmer einer Interaktion können während der **Interaktion erzeugt oder zerstört** werden.



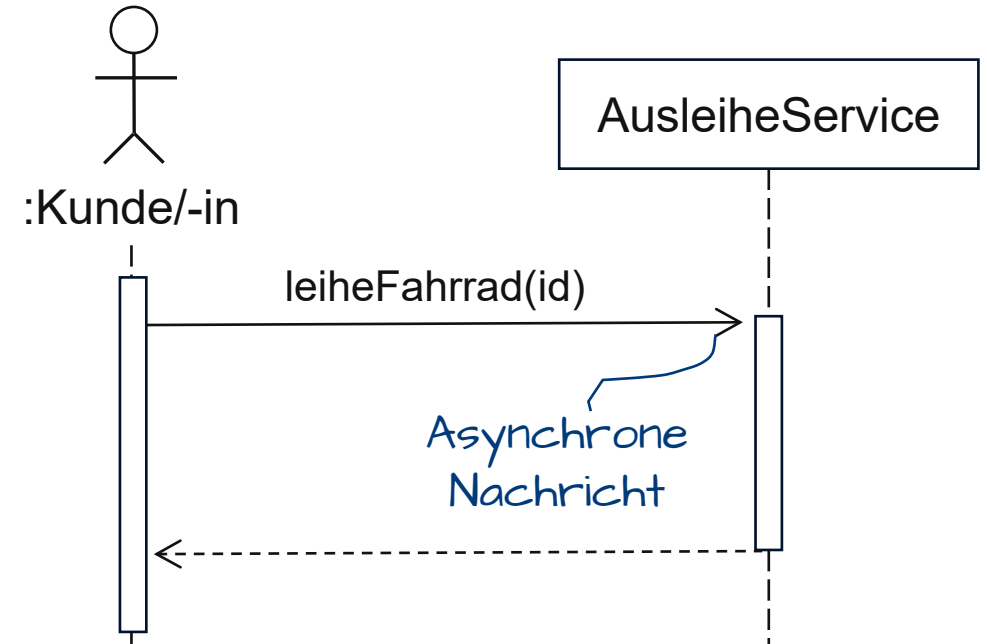
Synchrone und Asynchrone Nachrichten

Synchrone Nachrichten



Blockierend: Der Sender wartet auf eine Antwort des Empfängers, bevor er fortfährt.

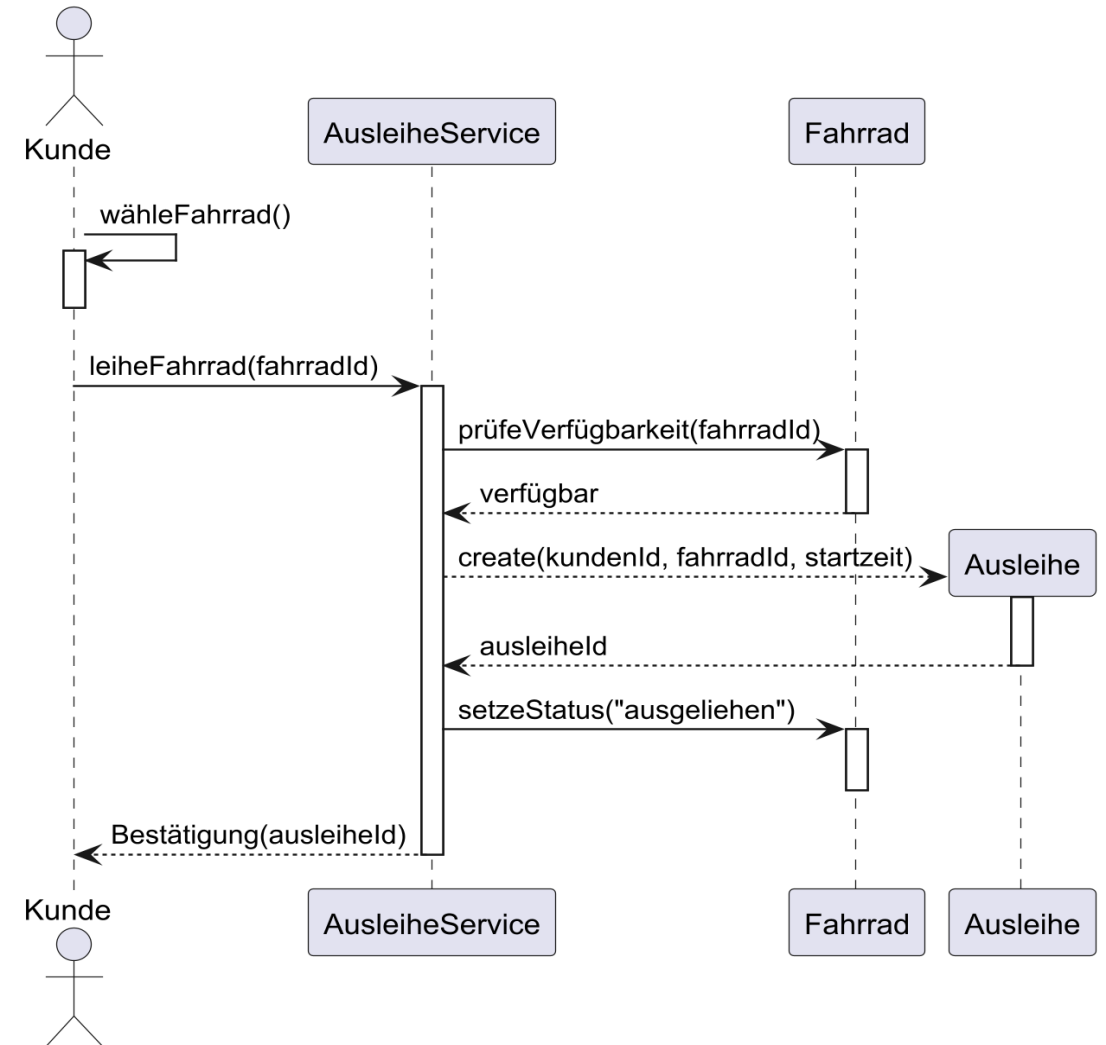
Asynchrone Nachrichten



Nicht-blockierend: Der Sender einer Nachricht fährt mit anderen Aufgaben fort, ohne auf eine Antwort zu warten.

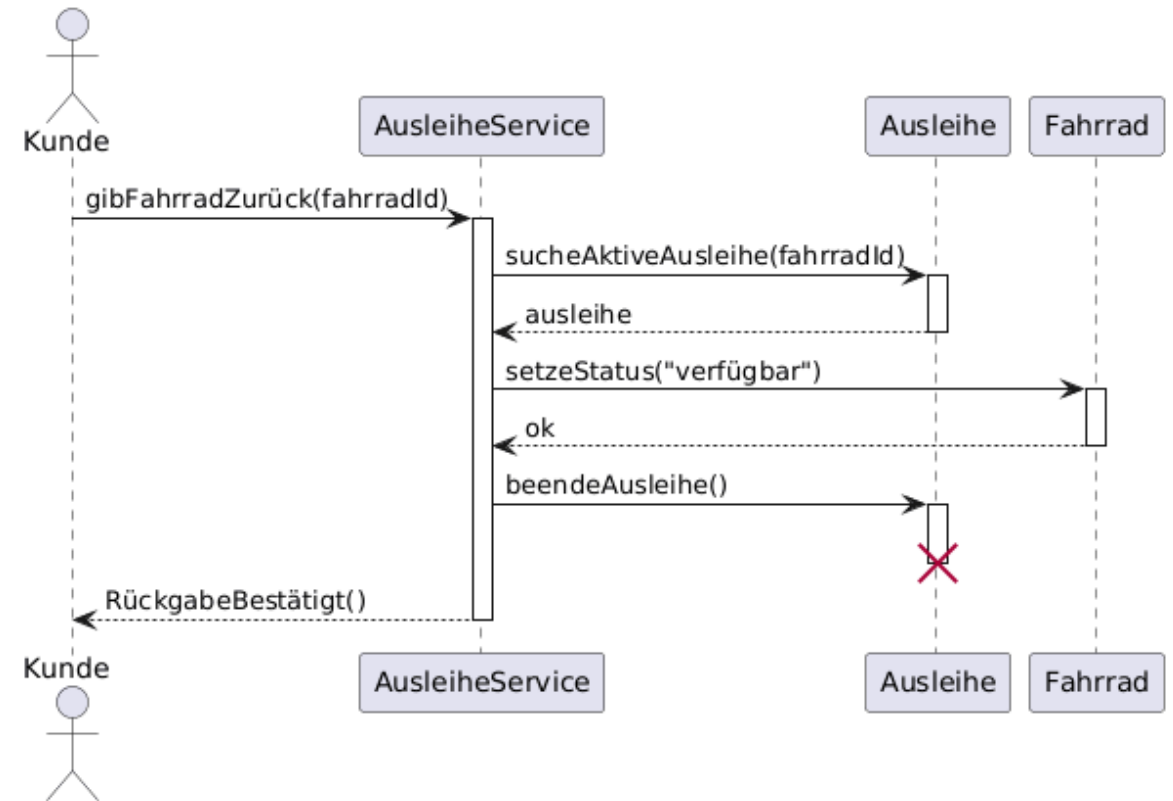
Aufgabe 3: Welche Aussagen treffen zu?

- A** Der/die Kund/-in führt zunächst einen Selbstaufruf aus.
- B** Die Instanz Ausleihe existiert bereits von Beginn des Diagramms an.
- C** ausleiheld wird als Antwortnachricht vom AusleiheService zurückgegeben.
- D** Das Fahrrad empfängt in diesem Diagramm genau eine Nachricht.
- E** Die Ausleihe wird nach Rückgabe des Fahrrads wieder zerstört



Aufgabe 4: Welche Aussagen treffen zu?

- A** Die Instanz Ausleihe wird in diesem Diagramm zerstört.
- B** Nach der Nachricht beendeAusleihe() kann Ausleihe keine weitere Nachricht mehr empfangen.
- C** RückgabeBestätigt() ist eine Antwortnachricht des Fahrrads.
- D** Das Diagramm enthält mindestens eine Antwortnachricht.
- E** Die Rückgabe des Fahrrads wird modelliert, ohne dass Informationen über das Fahrrad übergeben werden.





ÜBUNGSaufGABE 7.1

Bibliothekssystem: Buch ausleihen

Übungsaufgabe 7.1: Bibliothekssystem: Buch ausleihen

Aufgabenstellung

Modellieren Sie ein Sequenzdiagramm für folgenden Ablauf in einem Bibliothekssystem:

1. Ein/-e Nutzer/-in möchte ein Buch ausleihen.
2. Zunächst wählt der/die Nutzer/-in ein Buch aus. Anschließend sendet er/sie eine Anfrage an den Ausleihservice.
3. Der Ausleihservice prüft beim Buch, ob es verfügbar ist.
4. Ist das Buch verfügbar, erzeugt der Ausleihservice eine neue Ausleihe, setzt den Status des Buchs auf „ausgeliehen“ und bestätigt die Ausleihe gegenüber dem/der Nutzer/-in.

Das Diagramm soll mindestens enthalten:

- einen Selbstaufruf
- eine Nachricht mit Argument
- eine Antwortnachricht
- eine Erzeugungsnachricht



ÜBUNGSaufGABE 7.2

Online-Ticketbuchung: Kinoticket reservieren

Übungsaufgabe 7.2: Online-Ticketbuchung: Kinoticket reservieren

Aufgabenstellung

Modellieren Sie ein Sequenzdiagramm für folgenden Ablauf in einem Online-Kinobuchungssystem:

1. Ein/-e Kunde/-in möchte ein Ticket für eine Vorstellung reservieren.
2. Zunächst wählt der/die Kunde/-in intern eine Vorstellung aus. Danach sendet er/sie eine Reservierungsanfrage an das Buchungssystem.
3. Das Buchungssystem prüft beim Saal, ob noch ein Platz verfügbar ist.
4. Wenn ein Platz verfügbar ist, erzeugt das Buchungssystem eine Reservierung, markiert den Platz als reserviert und sendet eine Bestätigung an den/die Kunde/-in.

Das Diagramm soll mindestens enthalten:

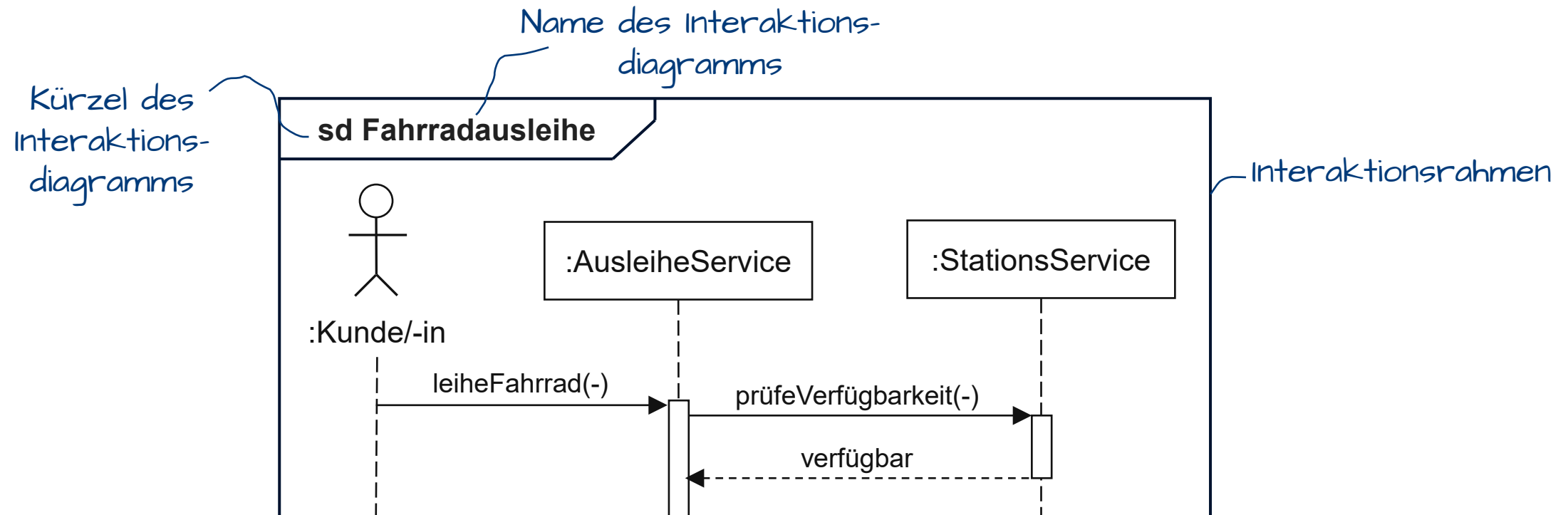
- einen Selbstaufruf
- mindestens zwei Nachrichten mit Argumenten
- mindestens eine Antwortnachricht
- eine Erzeugung eines neuen Objekts

UML-NOTATIONSELEMENT: INTERAKTIONSRAHMEN

Interaktionsrahmen



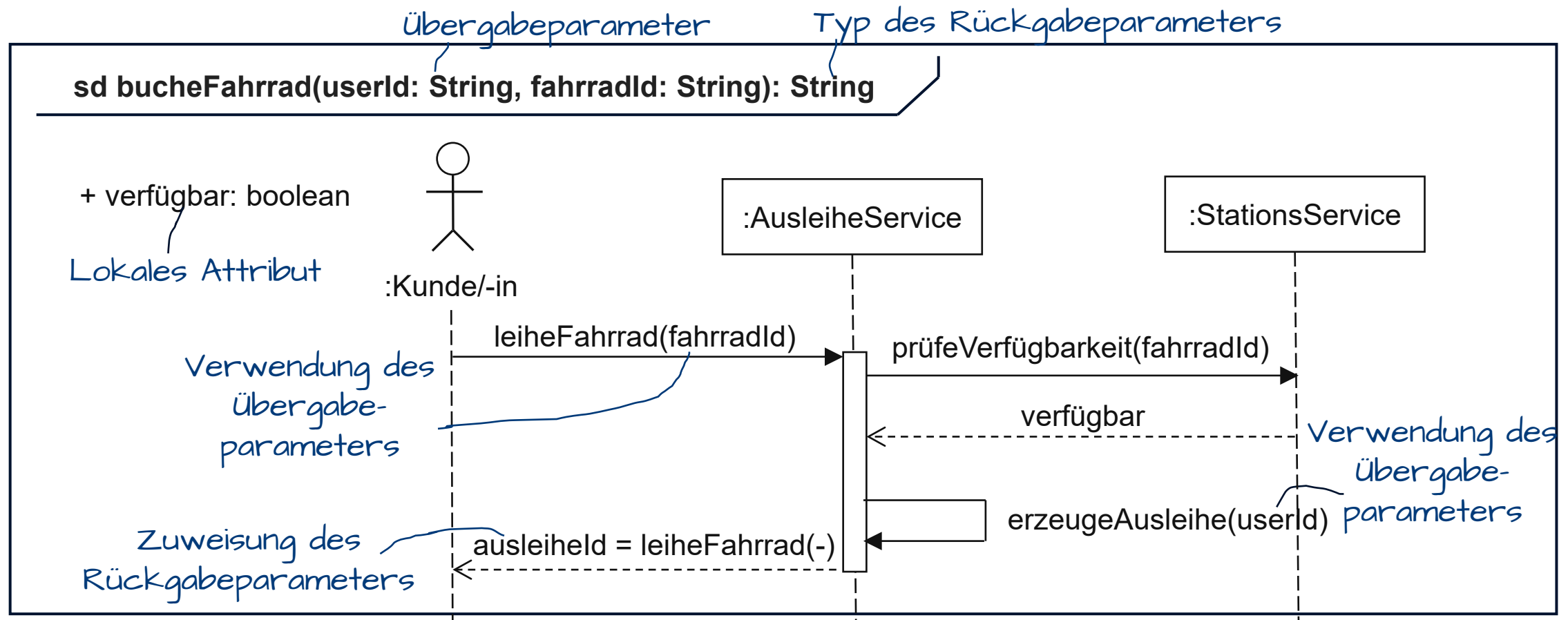
„Ein **Interaktionsrahmen** (engl. Interaction Frame) kapselt eine **Verhaltensdefinition**, deren Fokus auf der **Darstellung** eines **Informationsaustauschs** liegt.“ – Kecher*



Parameter und lokale Attribute innerhalb des Interaktionsrahmens nutzen



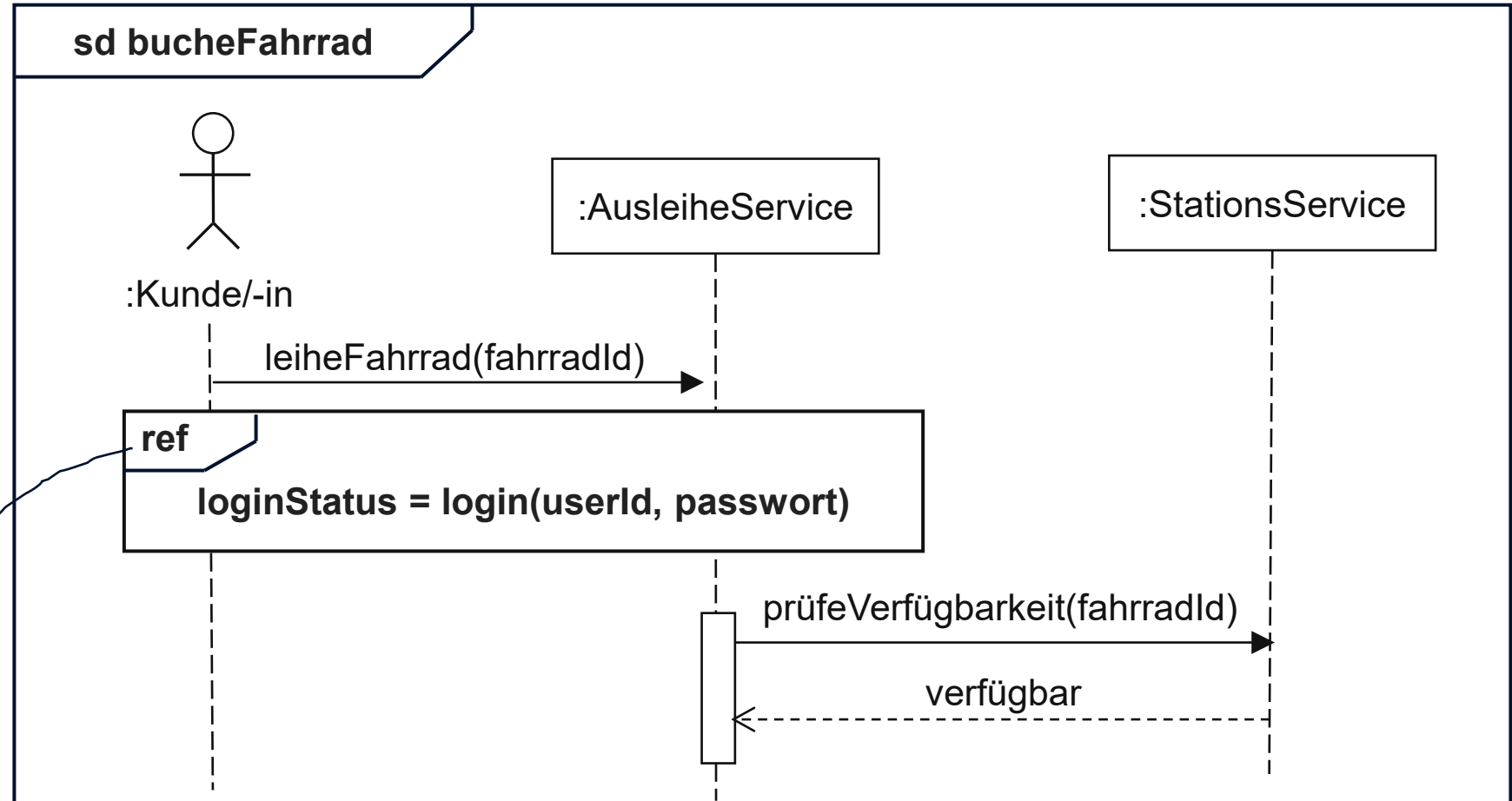
Eine **Interaktion** kann **Parameter empfangen** und **zurückgeben** sowie **lokal definierte Attribute** enthalten, die nur innerhalb der Interaktion verwendet werden.





Eine **Interaktionsreferenz** verweist auf eine **bereits definierte Interaktion** und ermöglicht es, diese an anderer Stelle **wiederverwenden**, anstatt sie erneut vollständig zu modellieren.

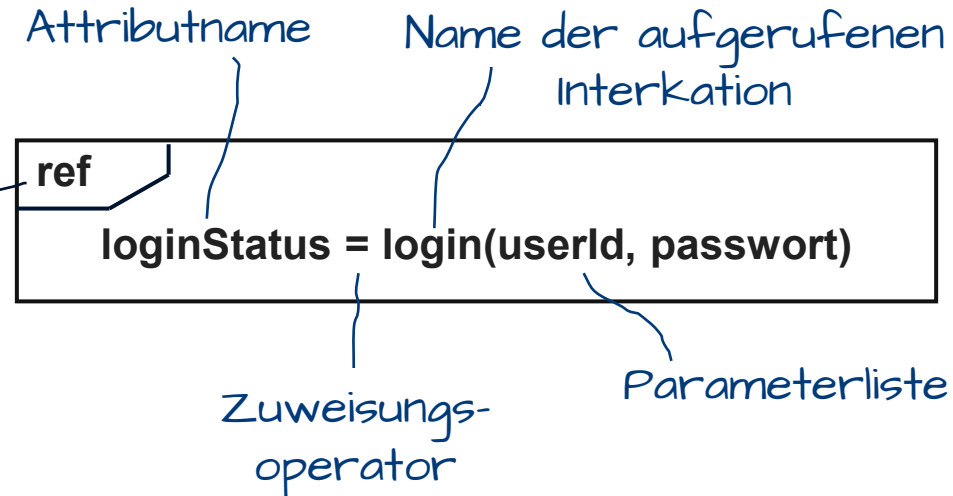
Interaktionsreferenzen
werden als ein
Interaktionsrahmen mit
der Typbezeichnung
ref notiert.

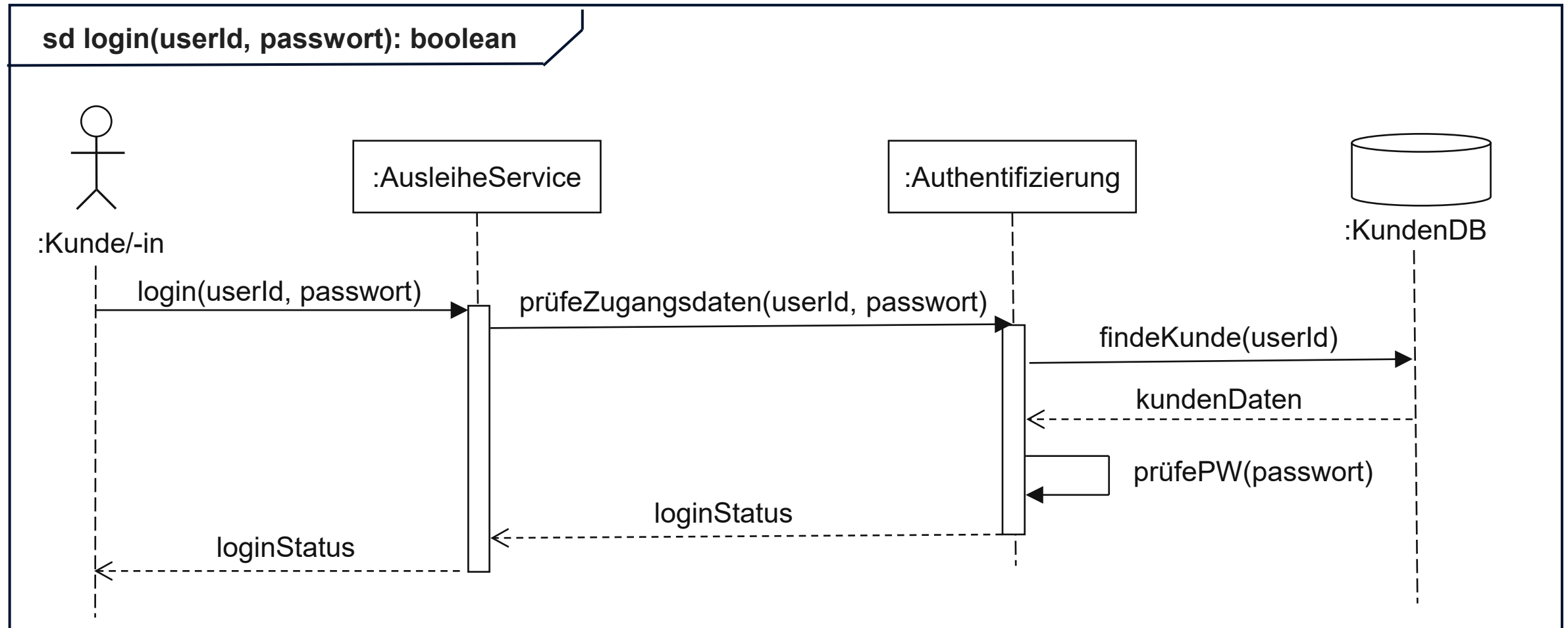




Eine **Interaktionsreferenz** verweist auf eine **bereits definierte Interaktion** und ermöglicht es, diese an anderer Stelle **wiederverwenden**, anstatt sie erneut vollständig zu modellieren.

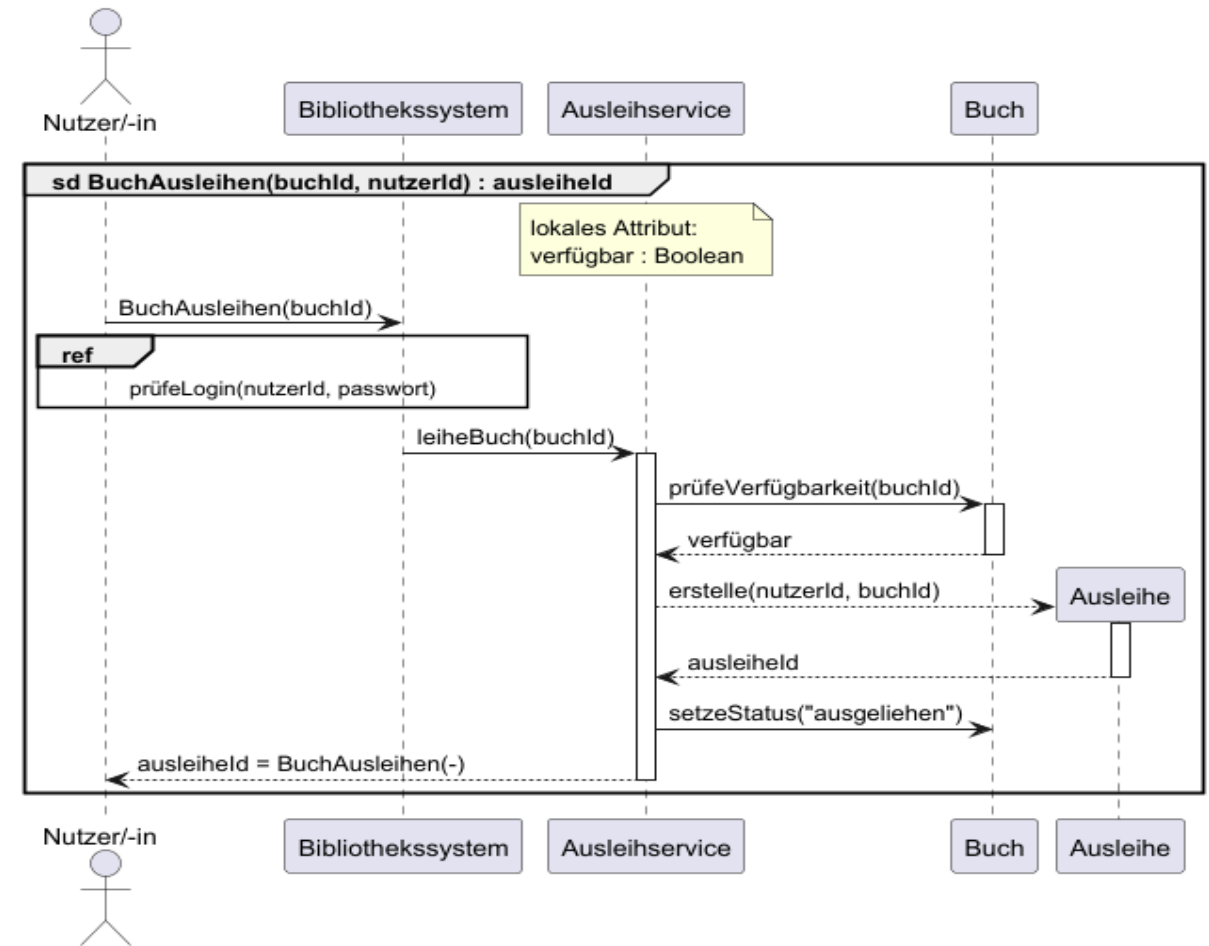
Interaktionsreferenzen werden als ein Interaktionsrahmen mit der Typbezeichnung **ref** notiert.





Aufgabe 5: Welche Aussagen treffen zu?

- A** Der Interaktionsrahmen besitzt zwei Übergabeparameter.
- B** ausleiheld ist ein Rückgabeparameter der Interaktion.
- C** verfügbar ist ein lokales Attribut innerhalb des Interaktionsrahmens.
- D** Die Interaktionsreferenz beschreibt eine separate Interaktion.
- E** verfügbar ist ein Rückgabeparameter der Interaktion.





ÜBUNGSAUFGABE 7.3

Modellierungsaufgabe – Restaurant: Gericht bestellen

Übungsaufgabe 7.3: Modellierungsaufgabe – Restaurant: Gericht bestellen

Aufgabenstellung

Modellieren Sie ein Sequenzdiagramm für die Interaktion „Gericht bestellen“ in einem Restaurant.

Ein Gast bestellt ein Gericht über ein Bestellsystem. Das Bestellsystem überprüft zunächst über eine Interaktionsreferenz *prüfeGericht(gerichtId)*, ob das gewünschte Gericht bestellt werden kann. Die Interaktion *prüfeGericht* prüft dabei zwei Aspekte:

1. Über die Speisekarte wird geprüft, ob das Gericht existiert.
2. Über das Lagersystem wird geprüft, ob die benötigten Zutaten verfügbar sind.

Erst wenn das Gericht bestellt werden kann, führt das Bestellsystem folgende Schritte aus:

1. Es wird eine neue Bestellung erzeugt.
2. Die Küche erhält den Auftrag, das Gericht zuzubereiten.
3. Das Bestellsystem bestätigt die Bestellung gegenüber dem Gast.

Übungsaufgabe 7.3: Modellierungsaufgabe – Restaurant: Gericht bestellen

Anforderungen

Ihr Sequenzdiagramm soll enthalten:

1. Interaktionsrahmen: sd GerichtBestellen(gerichtId, gastId) : bestellNr
2. Interaktionsreferenz: ref prüfeGericht(gerichtId)
3. Lokales Attribut z. B. verfügbar : Boolean

Weitere Anforderungen

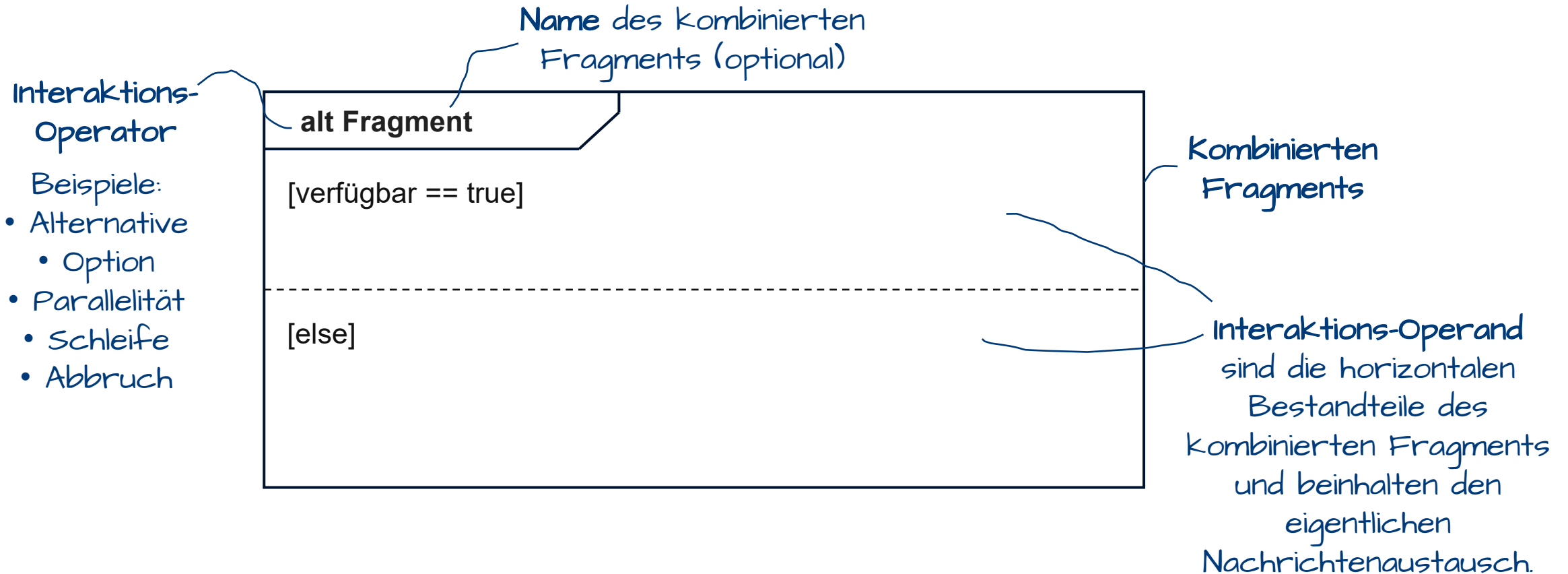
- mindestens eine Erzeugungsnachricht
- mindestens eine Antwortnachricht
- Nachrichten sollen in der Form Verb + Objekt (optional mit Argumenten) notiert werden.
- Beteiligte Teilnehmer: Gast, Bestellsystem, Speisekarte, Lagersystem, Küche, Bestellung

UML-NOTATIONSELEMENT: KOMBINIERTE FRAGMENTE

Kombinierte Fragmente



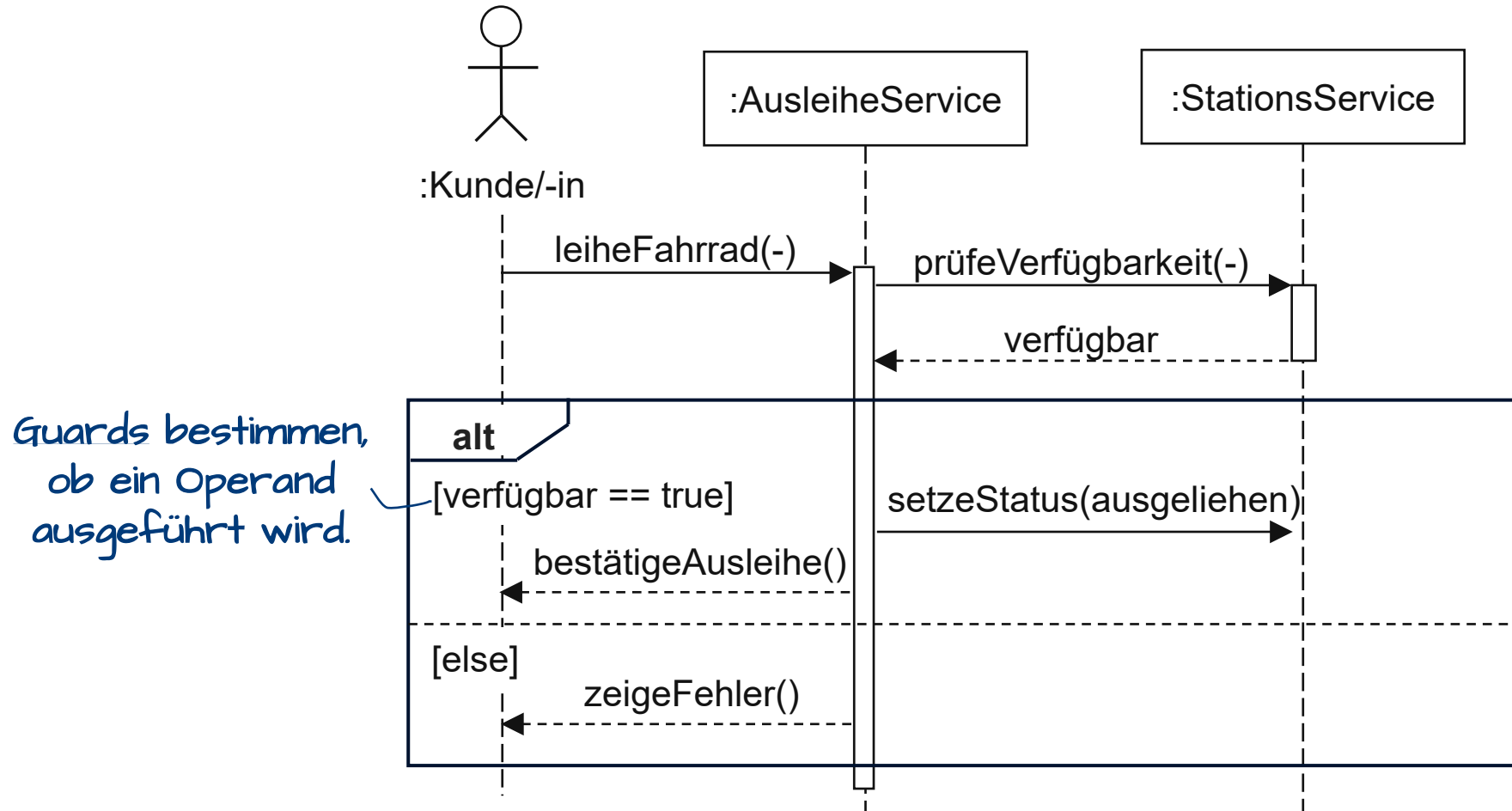
Kombinierte Fragmente beschreiben **Kontrollstrukturen** innerhalb von **Interaktionen**. Sie steuern, wann und wie Nachrichten ausgeführt werden.



Kombiniertes Fragment: Alternative



„In einem mit **alt** gekennzeichneten kombinierten Fragment darf nur **einer der Interaktions-Operanden ausgeführt** werden. Er entspricht einem **if-then-else-Konstrukt**.“ – Kecher*



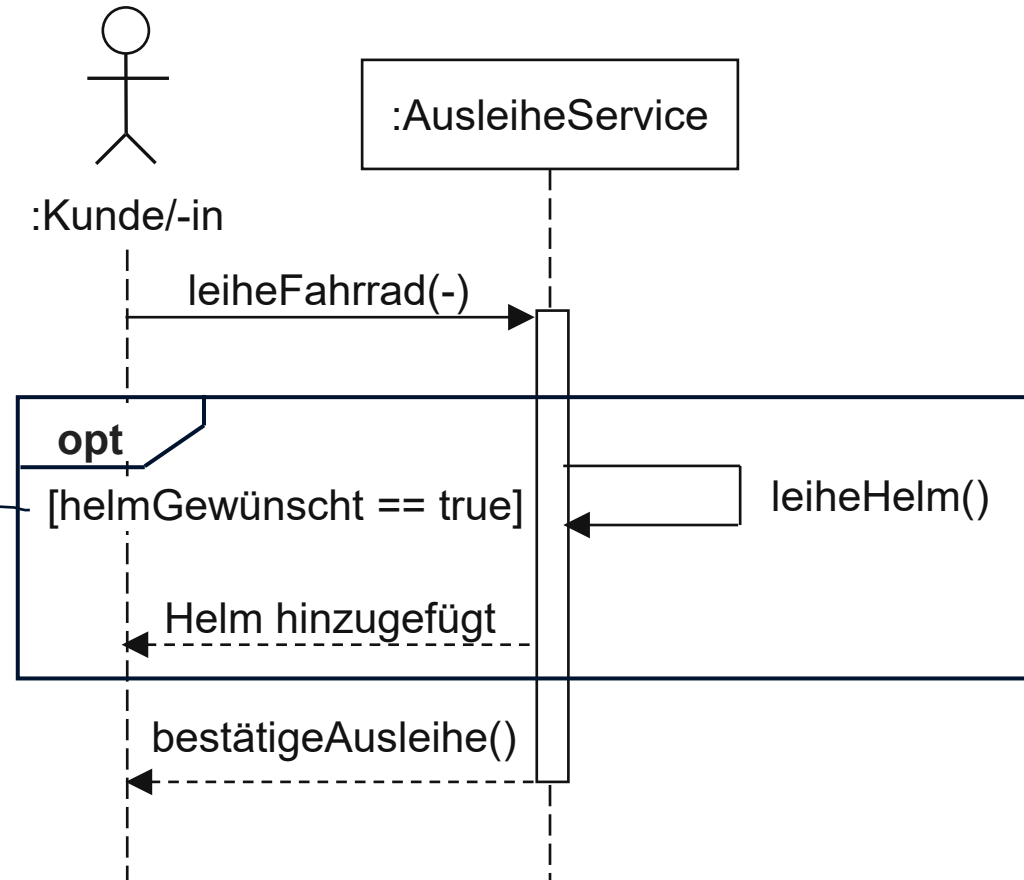
UML schreibt vor, dass höchstens ein Operand ausgeführt wird. Daher müssen die Guards disjunkt sein, sich also gegenseitig vollständig ausschließen.

Kombiniertes Fragment: Option



„Durch den Interaktions-Operator **opt** wird ein kombiniertes Fragment als **optional** gekennzeichnet.“ – Kecher*

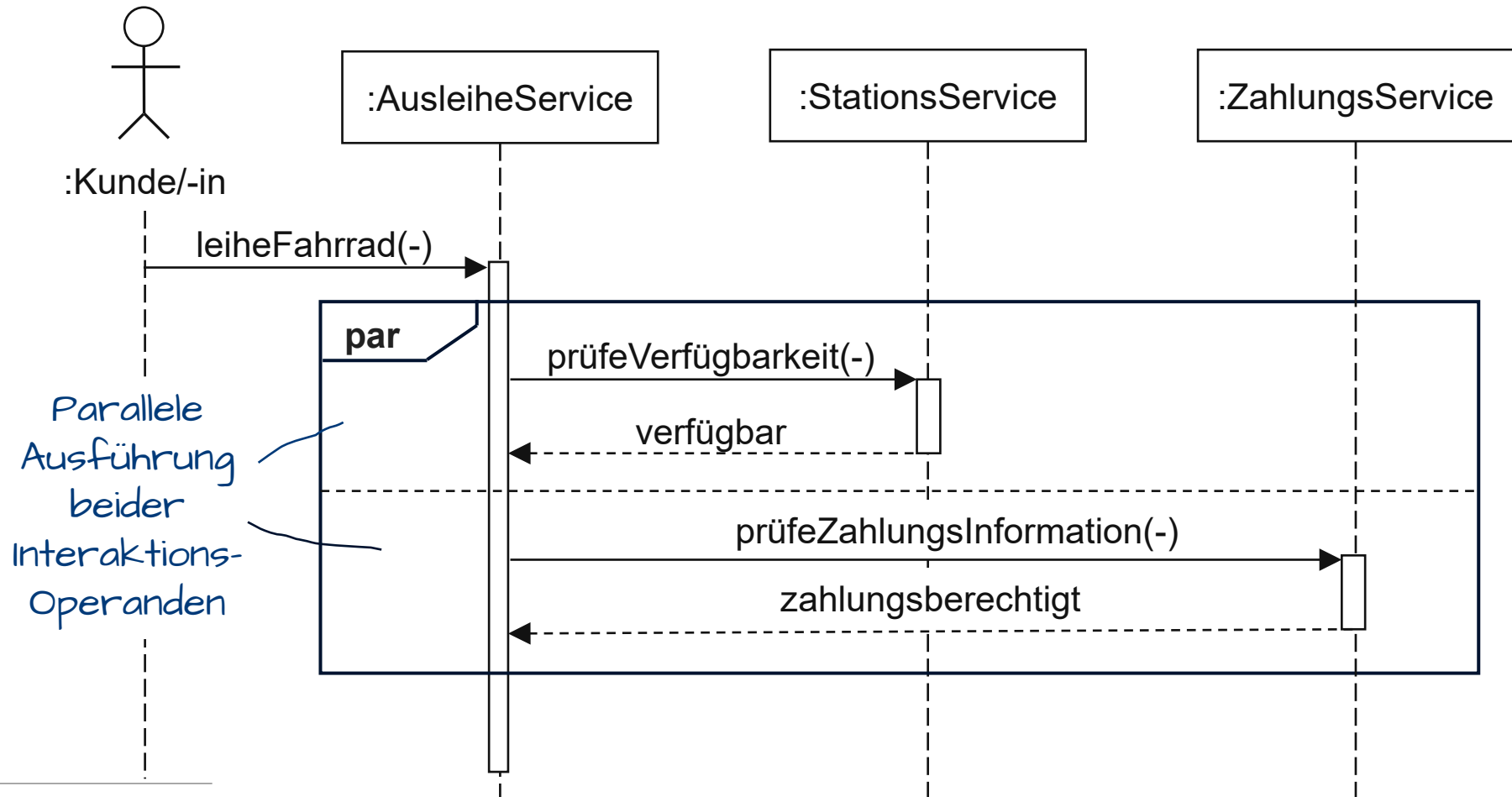
Ein **opt**-Fragment wird nur bei erfülltem Guard ausgeführt und entspricht einem **alt**-Fragment mit nur einem Operanden.



Kombiniertes Fragment: Parallelität



„Der Interaktions-Operator **par** kennzeichnet die Ausführung der Interaktions-Operanden als **parallel**.“ – Kecher*

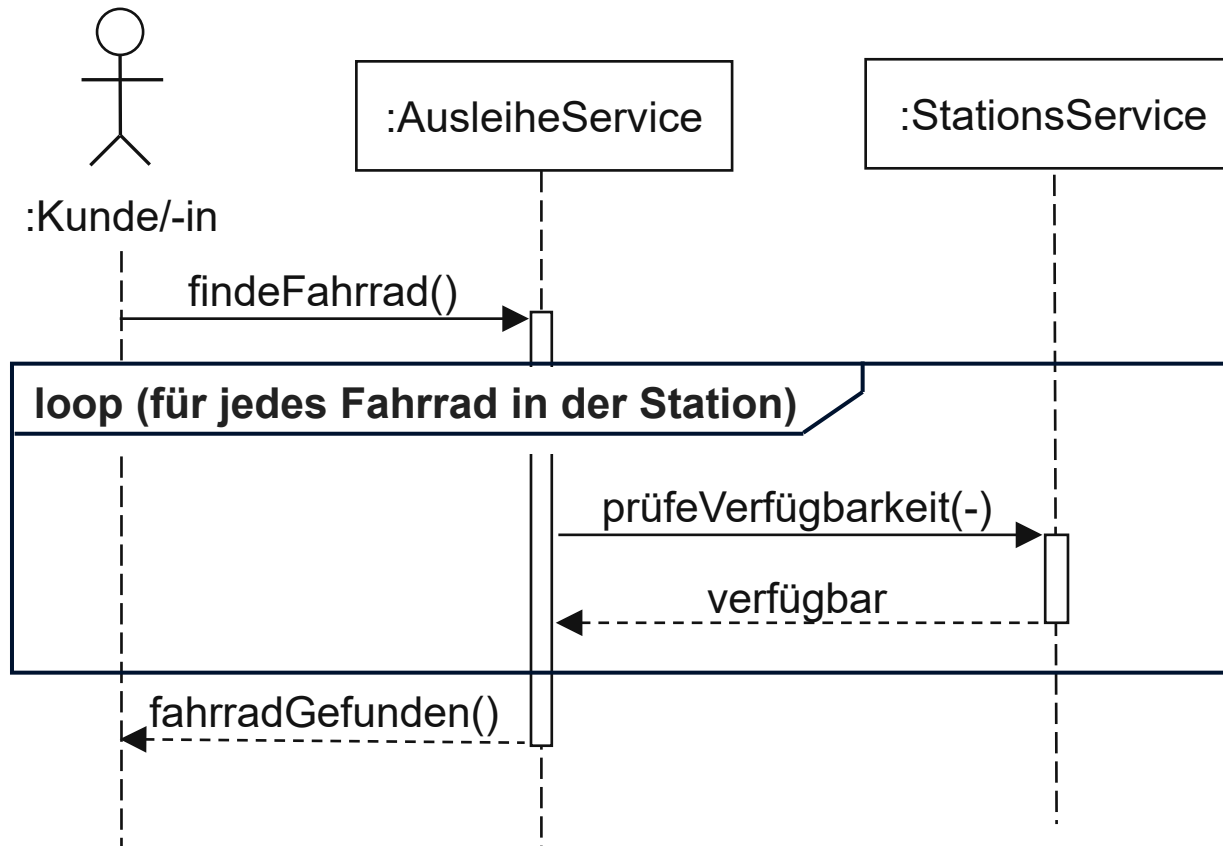


*Quelle: UML 2.5 - Das umfassende Handbuch, Kecher, Hoffmann-Elbern, Will, Rheinwerk, 2021

Kombiniertes Fragment: Schleife



„Der **Schleifen-Operator loop** spezifiziert eine Anzahl von **wiederholten Ausführungen** des Interaktions-Operanden.“ – Kecher*



Syntax des Schleifen-Operators

```
loop [(<minint> [, <maxint> ])]
```

```
loop [(<boolescher Ausdruck>)]
```

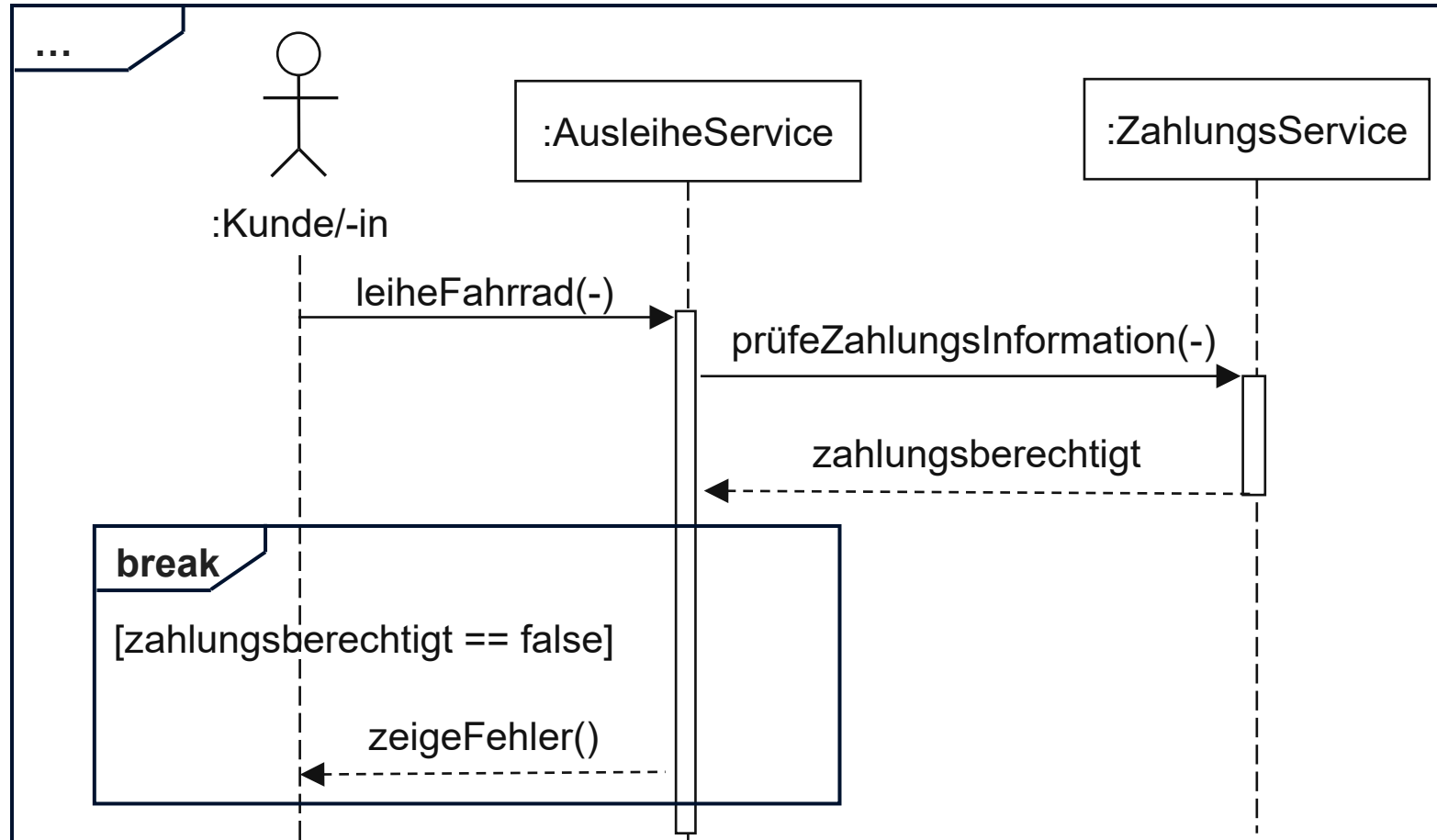
Beispiele

- `loop(4)` - 4 Wiederholungen
- `loop(2, *)` - minimal 2 bis beliebig viele Wiederholungen
- `loop` oder `loop(*)` - beliebig viele Wiederholungen
- `loop(2, 4)` - 2 bis 4 Wiederholungen
- `loop(x < 3)` - Schleife wird so lange wiederholt bis der Ausdruck zu false wird

Kombiniertes Fragment: Abbruch



„Mit dem Abbruch-Operator **break** wird ein kombiniertes Fragment zur Behandlung von **Abbruchsituationen** definiert.“ – Kecher*

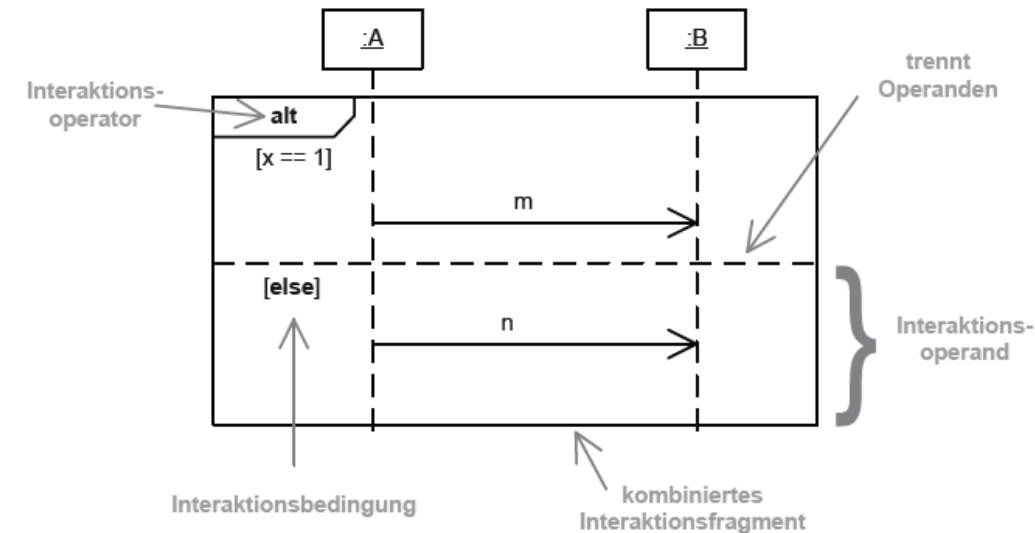


Wird der Guard eines break-Fragments erfüllt, wird der Operand ausgeführt und das umgebende Fragment anschließend abgebrochen.

Sequenzdiagramme – Kombinierte Fragmente

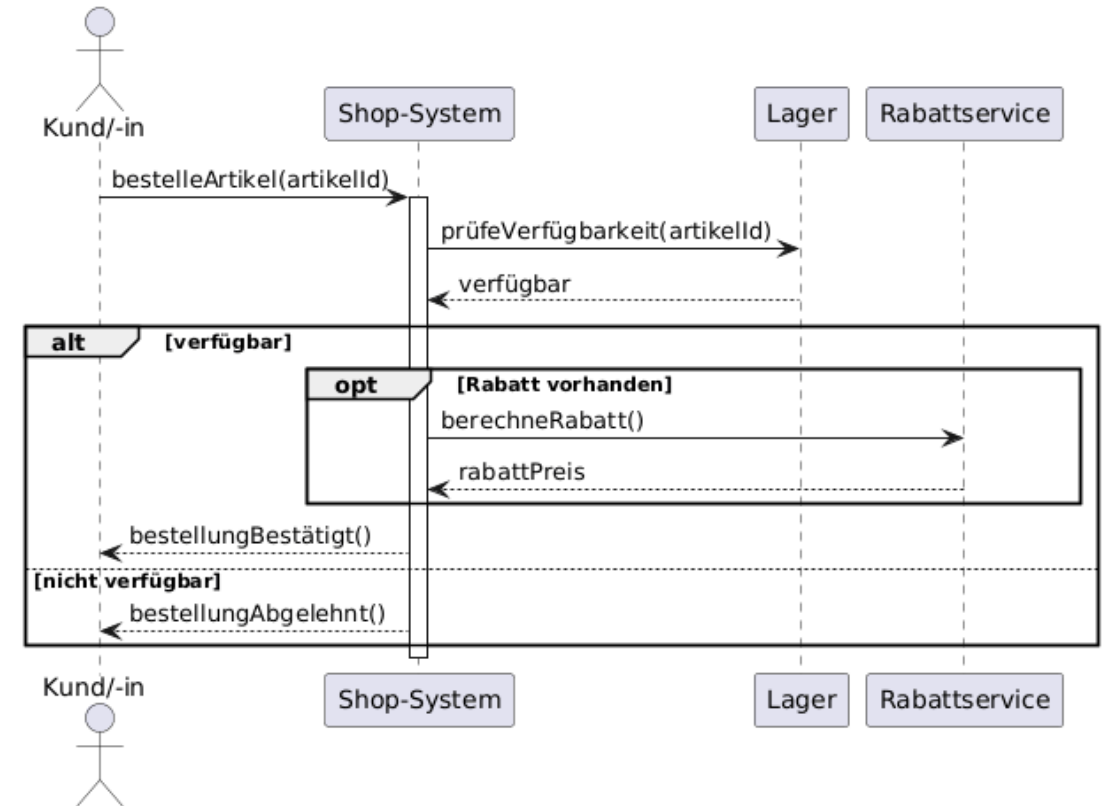
- Es gibt eine Reihe weiterer Fragmenttypen

Deutsche Bezeichnung	Englische Bezeichnung	Kürzel im Diagramm	Damit modellieren Sie ...
Alternative Fragmente	Alternative	alt	... alternative Ablaufmöglichkeiten
Optionales Fragment	Option	opt	... optionale Interaktionsteile
Abbruchfragment	Break	break	... Interaktionen in Ausnahmefällen
Negation	Negative	neg	... ungültige Interaktionen
Schleife	Loop	loop	... iterative Interaktionen
Parallele Fragmente	Parallel	par	... nebenläufige Interaktionen
Lose Ordnung	Weak Sequencing	seq	... von Lebenslinie und Operanden <i>abhängige</i> chronologische Abläufe
Strenge Ordnung	Strict Sequencing	strict	... von Lebenslinie und Operanden <i>unabhängige</i> chronologische Abläufe
Kritischer Bereich	Critical Region	critical	... atomare Interaktionen
Irrelevante Nachrichten	Ignore	ignore	... Filter für unwichtige Nachrichten
Relevante Nachrichten	Consider	consider	... Filter für wichtige Nachrichten
Sicherstellung	Assertion	assert	... unabdingbare Interaktionen



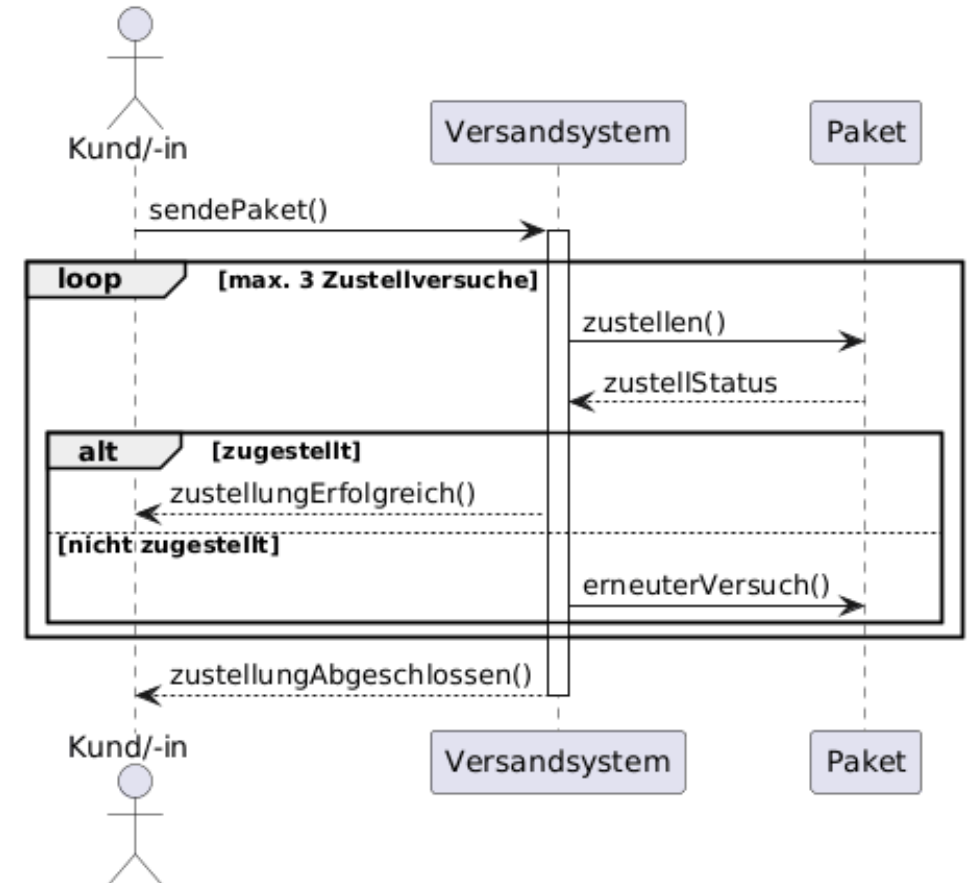
Aufgabe 6: Welche Aussagen treffen zu?

- A** Das Diagramm enthält sowohl ein alt- als auch ein opt-Fragment.
- B** Der Rabatt wird immer berechnet.
- C** Die Bestellung wird nur bestätigt, wenn der Artikel verfügbar ist.
- D** Das opt-Fragment wird nur im ersten Operanden des alt-Fragments ausgeführt.
- E** Wenn kein Rabatt vorhanden ist, wird die Bestellung abgelehnt.



Aufgabe 7: Welche Aussagen treffen zu?

- A** Das Diagramm enthält ein loop-Fragment.
- B** Die Zustellung wird nur einmal versucht.
- C** Das alt-Fragment entscheidet über den weiteren Ablauf innerhalb der Schleife.
- D** Bei erfolgreicher Zustellung wird kein weiterer Zustellversuch durchgeführt.
- E** Das Diagramm enthält sowohl Schleifen- als auch Alternativlogik.





ÜBUNGSAUFGABE 7.4

Modellierungsaufgabe – Sortieralgorithmus

Übungsaufgabe 7.4: Modellierungsaufgabe – Sortieralgorithmus

Aufgabenstellung

Modellieren Sie ein Sequenzdiagramm für einen vereinfachten Bubble-Sort-Algorithmus, der eine Liste von Zahlen sortiert. Ein/-e Nutzer/-in startet den Sortiervorgang. Der Sortieralgorithmus führt anschließend mehrere Durchläufe über die Liste aus. In jedem Durchlauf werden benachbarte Elemente der Liste betrachtet.

Der Ablauf eines Vergleichsschritts ist wie folgt:

1. Der Sortieralgorithmus fordert die Liste auf, zwei benachbarte Elemente zu liefern.
2. Die Liste gibt die beiden Werte zurück.
3. Der Sortieralgorithmus vergleicht die beiden Werte.
4. Wenn das erste Element größer als das zweite ist, weist der Algorithmus die Liste an, die beiden Elemente zu vertauschen.

Dieser Vorgang wird für mehrere Elementpaare der Liste wiederholt, bis die Liste vollständig durchlaufen wurde. Am Ende meldet der Sortieralgorithmus dem/der Nutzer/-in, dass die Sortierung abgeschlossen ist.

Übungsaufgabe 7.4: Modellierungsaufgabe – Sortieralgorithmus

Anforderungen

Ihr Sequenzdiagramm soll enthalten:

- mindestens drei Teilnehmer
- eine Schleife für die wiederholten Vergleiche
- mindestens eine Nachricht mit Argumenten
- mindestens eine Antwortnachricht
- einen Selbstaufruf des Algorithmus

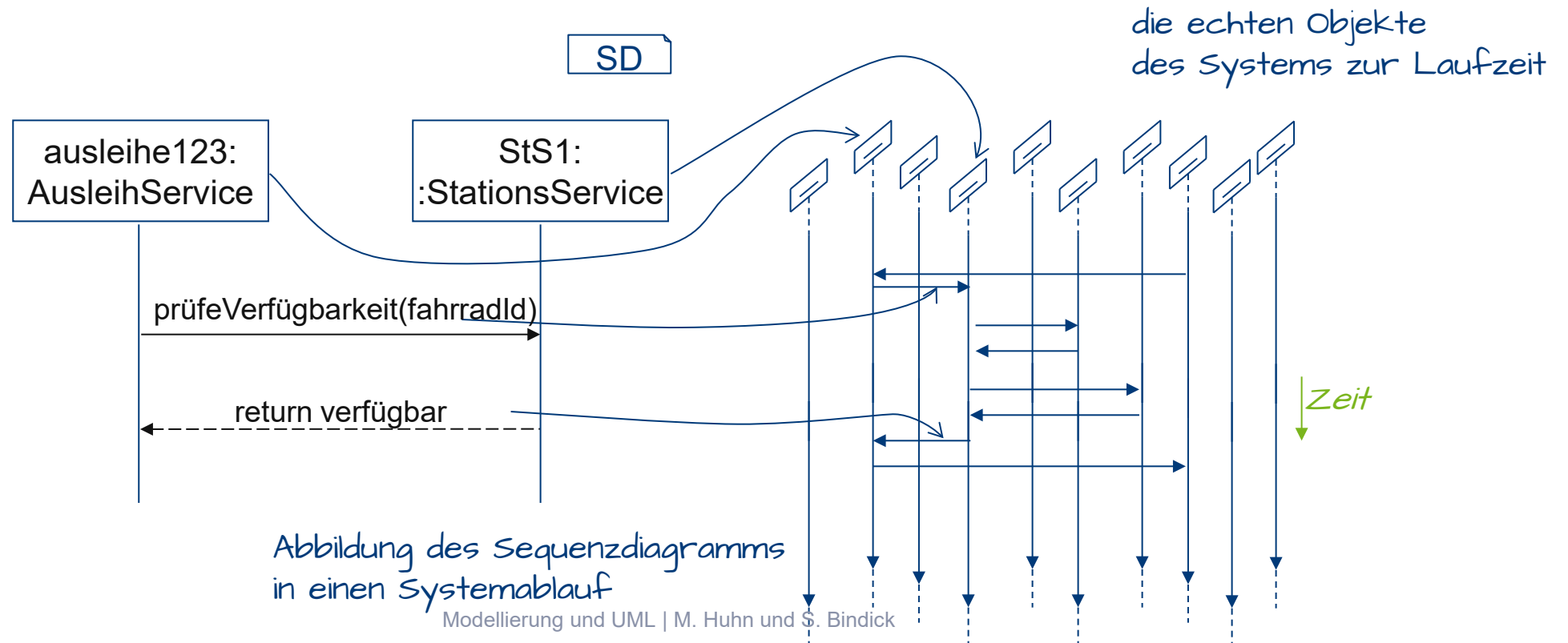
SEMANTISCHE DETAILS

Exemplarität und Unvollständigkeit

- Ein Sequenzdiagramm beschreibt **einen Ausschnitt eines Ablaufs des Systems**:
 - Die Objektmenge ist in der Regel unvollständig
 - Argumente von Methodenaufrufen dürfen fehlen
 - Vor und nach dem gezeigten SD finden weitere Interaktionen statt.
 - Was ist mit zwischendurch?
 - Der dargestellte Ablauf kann mehrfach auftreten,
 - er kann zeitlich verschachtelt auftreten,
 - er kann auch gar nicht auftreten.
- Welche Semantik hat nun ein Sequenzdiagramm?

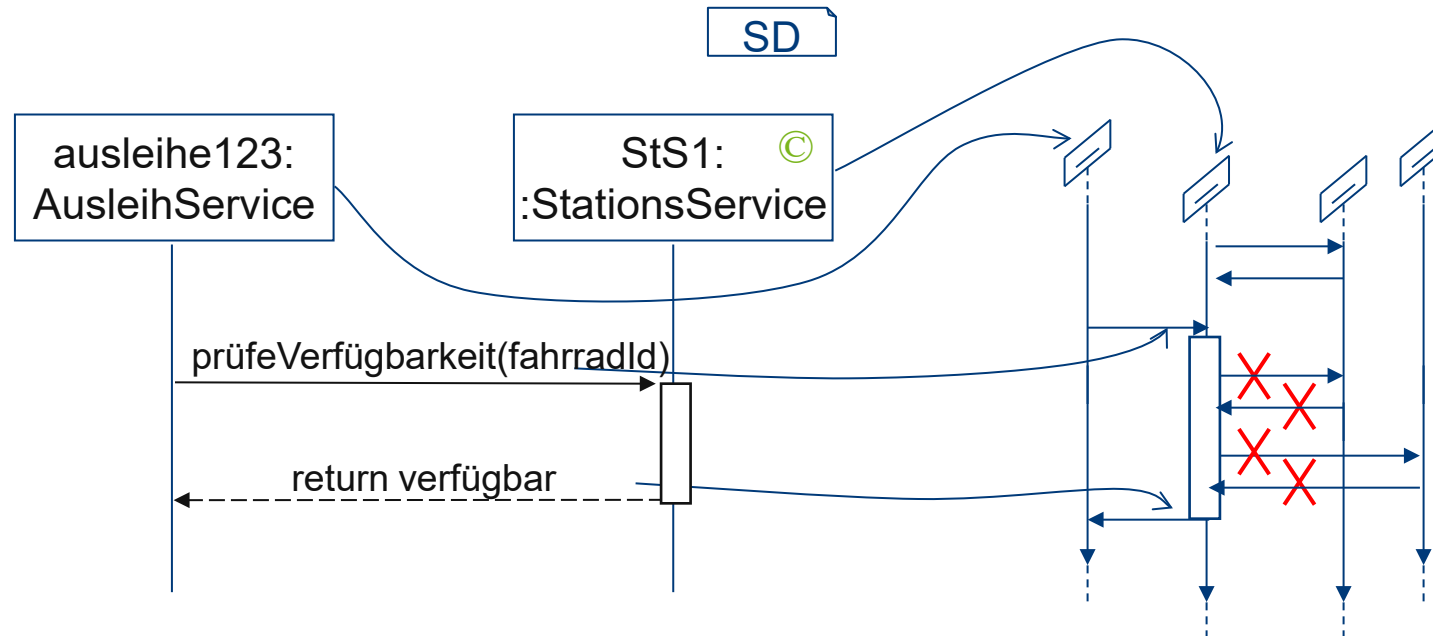
Präzise Bedeutung eines SD

- Durch Abbildung der
 - prototypischen Objekte im SD auf echte Objekte des Systems
 - Interaktionen des SD auf echte Interaktionen im System
 - (präzise Definition siehe z.B. B. Rumpe: Modellierung mit UML)



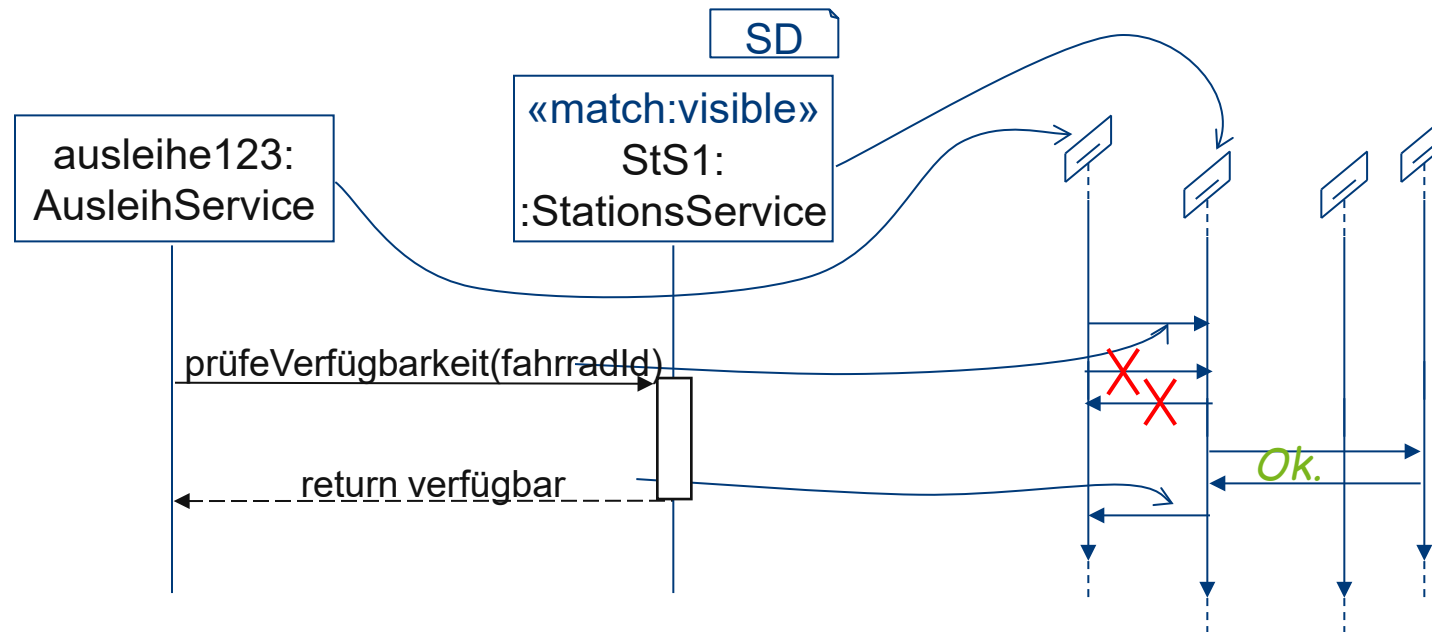
Variante: Vollständige Interaktionsliste

- Beispiel: Alle im Zeitraum der Beobachtung stattfindenden Interaktionen eines Objekts sind enthalten, d.h. es gibt keine weiteren.
- Markierung «match:complete» (kurz: ©) verbietet andere Interaktionen zwischendurch:



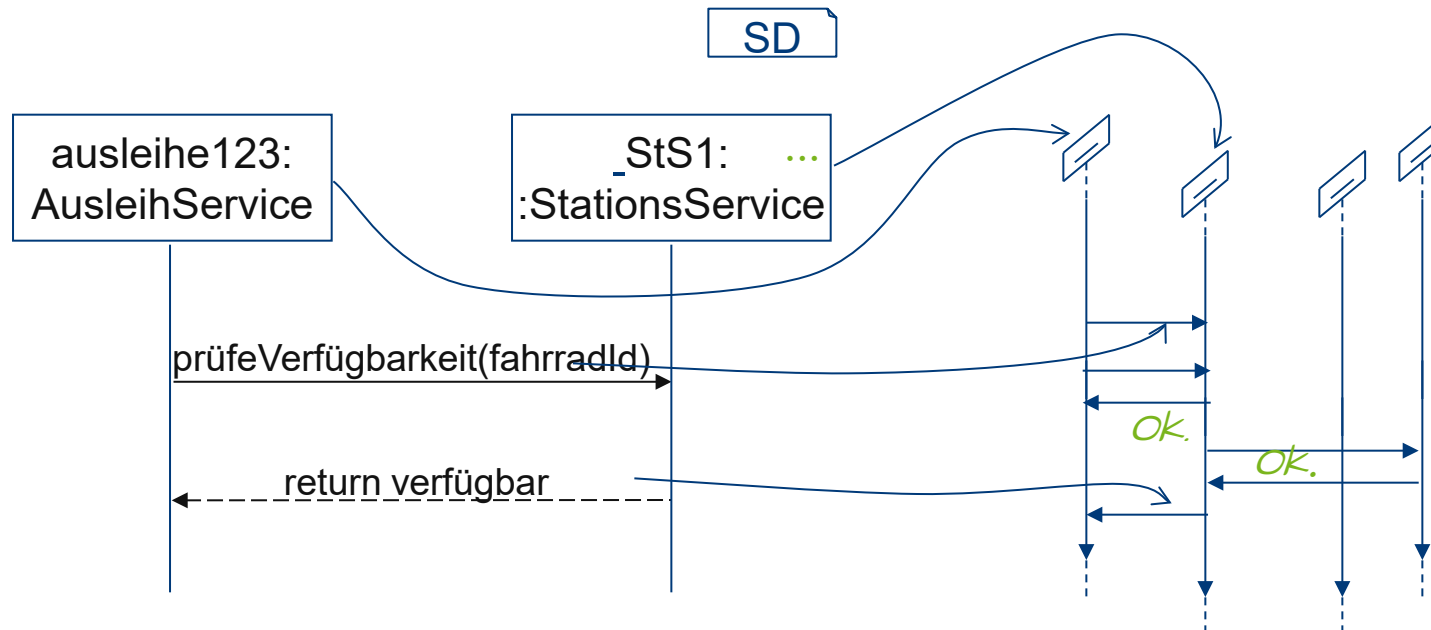
Variante: Fast vollständige Interaktionsliste

- Beispiel: Alle im Zeitraum der Beobachtung stattfindenden Interaktionen eines Objekts mit anderen im Sequenzdiagramm sichtbaren Objekten sind enthalten, d.h. es gibt keine weiteren.
- Markierung «match:visible» verbietet andere Interaktionen zwischendurch mit sichtbaren Objekten:



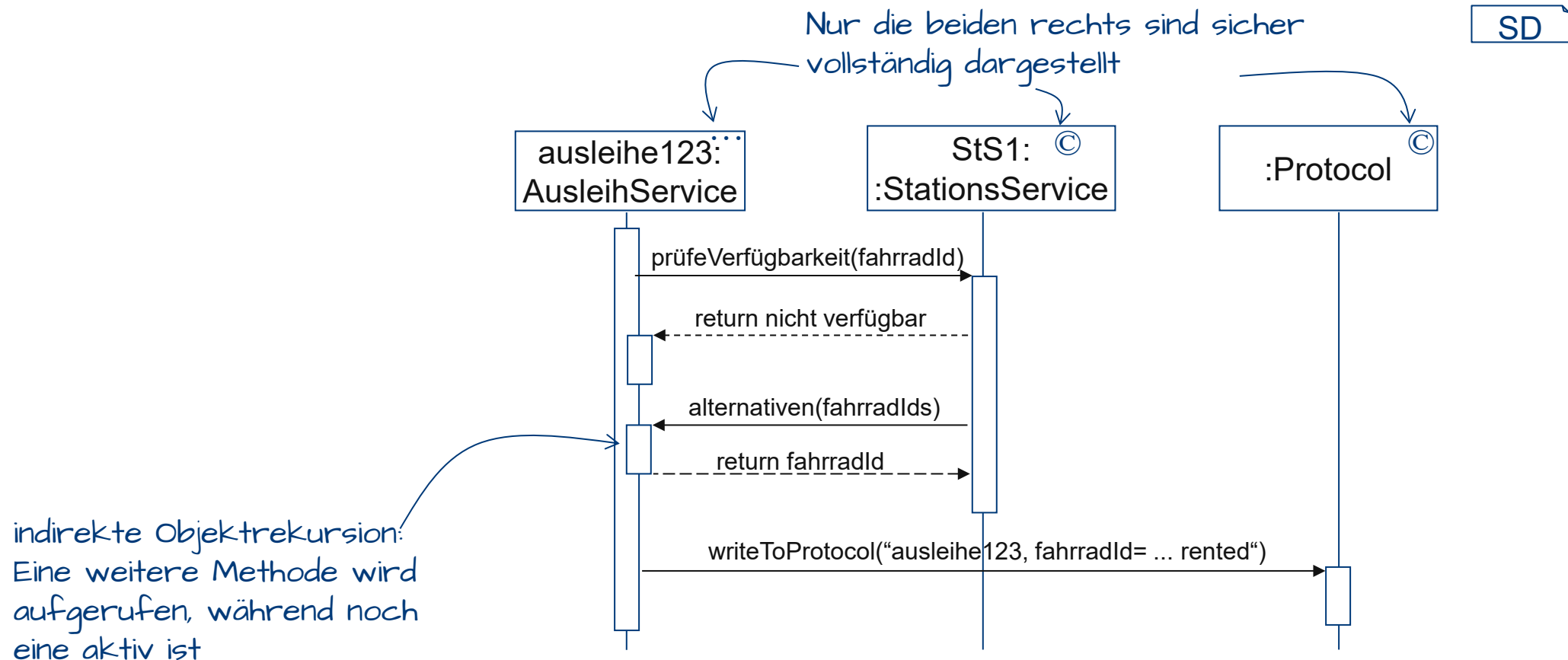
Variante: Unvollständige Interaktionsliste

- Beispiel: Beliebige andere Interaktionen zwischen dargestellten Objekten sind möglich
- Markierung **«match:free»** (kurz: ...) erlaubt beliebige andere Interaktionen auch zwischendurch



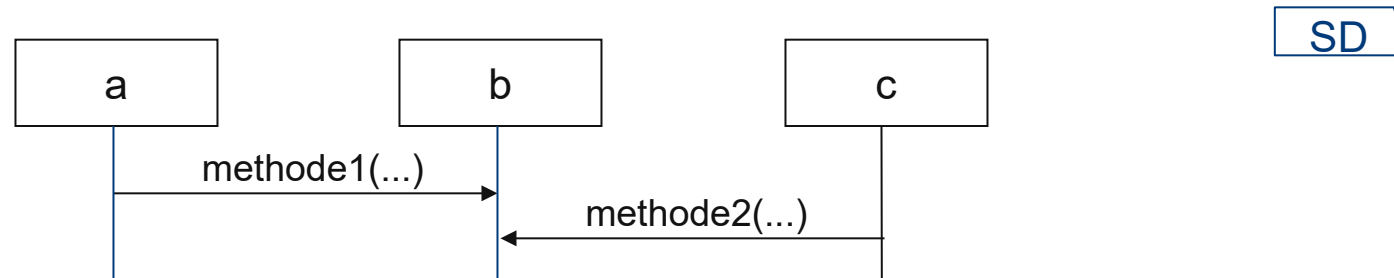
Kombinierbarkeit der Stereotypen

- Erlaubt die Konzentration der Modellierung auf die unter Beobachtung befindlichen Kern-Objekte:



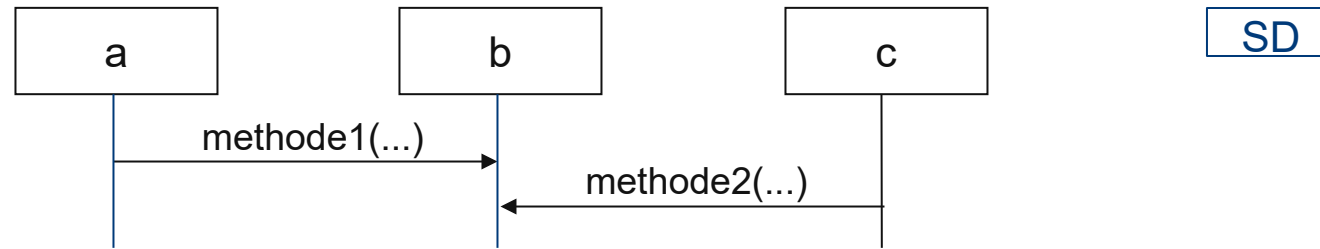
Spezialfälle und deren Semantik ...

- **Nicht-kausales SD** ist ein SD, bei dem die Wirkzusammenhänge (Kausalität) nicht geklärt sind.
- Ist das SD sinnvoll? Was bedeutet dieses SD?



Spezialfall: nicht-kausales SD

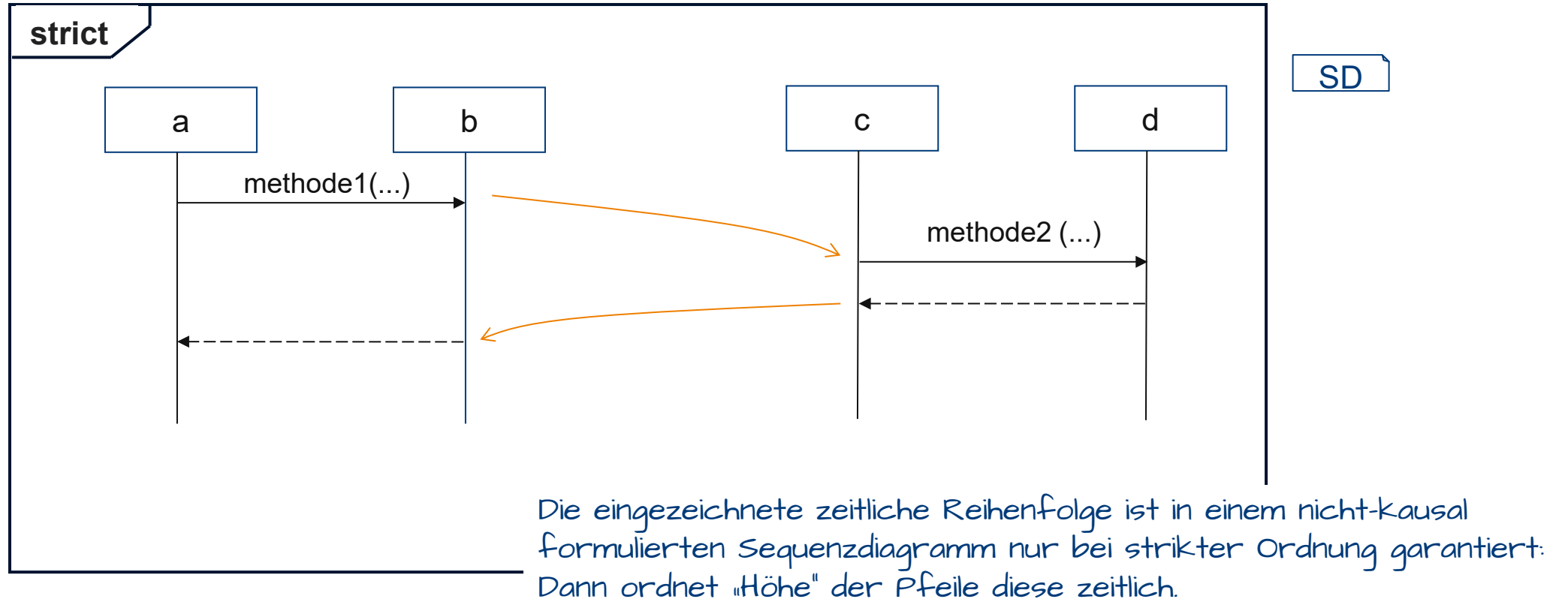
- Nicht kausales SD ist ein SD, bei dem die Wirkzusammenhänge (Kausalität) nicht geklärt sind.



- Bedeutung:
 - nicht-kausale aber **mögliche Beobachtung**, zum Beispiel aufgrund eines nicht angegebenen Aufrufs von a nach c (oder von b nach c)
- SD ist nicht zur konstruktiven Codegenerierung (der Methodenrumpfe) verwendbar, da essentielle Information fehlt

Spezialfall: Unzusammenhängendes SD

- Weitere Form eines nicht-kausalen SD:

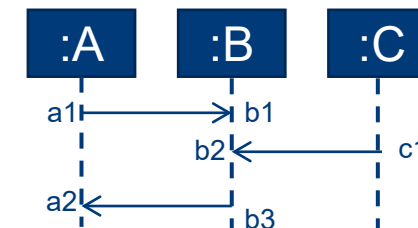
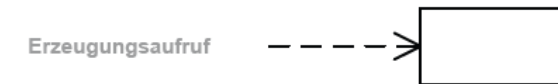


Es bleibt die Frage, wie das System gewährleisten wird, dass methode1 von b bearbeitet wird, bevor c methode2 aufruft

Zusammenfassung: Reihenfolge von Nachrichten

- Unterscheidung synchrone (Sender blockiert) und asynchrone Nachrichten (Sender arbeitet weiter)
- Regeln für zeitlichen Ablauf: lose Ordnung (weak sequencing) als default
 - Chronologische Reihenfolge auf eine Lebenslinie
 - Reihenfolge der Ereignisse auf unterschiedlichen Lebenslinien ist unabhängig
 - Sendeereignis vor Empfangsereignis
- Falls strenge Reihenfolge der Ereignisse benötigt wird: strict-Operator

	Sende- und Empfangsereignis treten auf	Sendeereignis ist nicht bekannt	Empfangsereignis ist nicht bekannt
synchroner Operationsaufruf			
asynchroner Signal-/Operationsaufruf			
Antwortnachricht/ Rücksprung auf einen synchronen Operationsaufruf			



Ordnung:
 a1 vor a2
 a1 vor b1
 b1 vor b2 vor b3
 c1 vor b2
 b3 vor a2

Mögliche Beobachtungen (traces):

a1 b1 c1 b2 b3 a2
 c1 a1 b1 b2 b3 a2
 a1 c1 b1 b2 b3 a2

ZUSAMMENFASSUNG

Zweck

Sequenzdiagramme beschreiben, wie Instanzen zeitlich geordnet miteinander interagieren.

Zentrale Elemente

- Lebenslinien → beteiligte Instanzen
- Nachrichten → Kommunikation (Operationsaufrufe, Signale)
- Aktivierungen → Ausführungsphasen
- Antwortnachrichten → Rückgaben von Operationen

Erweiterungen

- Zustandsinvarianten → Zustand eines Objekts zu einem Zeitpunkt
- Erzeugung / Destruktion → Objekte entstehen und enden während der Interaktion
- Selbstaufrufe → interne Aktivitäten eines Teilnehmers

Strukturierung

- Interaktionsrahmen → kapseln Interaktionen (Parameter, Rückgabe, lokale Attribute)
- Interaktionsreferenzen (ref) → Wiederverwendung von Abläufen

Kontrollstrukturen (kombinierte Fragmente)

- alt → Alternative (if / else)
- opt → optional (if)
- loop → Wiederholung
- par → Parallelität
- break → Abbruch